Pockit

CPE 334, Project Deliverable 1

Engineering baseline and Sprint 1 results

Theme 7: Personal Money and Expense Logging System

Ready for final team review

This self-contained report includes Parts A to F, stable IDs, test and demo evidence, contribution and AI-use records, and links to detailed project history. Both students should verify the final copy before submission.

Student 1	Guntee Doungmanee, student number 67070501003, @ovenmakemeheat
Student 2	Polwarit Watthanahemmarat, student number 67070501067, @MadMax168

Repository	Pockit on GitHub
Release tag	`sprint1-v1.0.0`
Release commit	`e99da77c8181a97a32abfddc37b116ff297acf27` on `main`
Assignment	Project Deliverable 1 specification
Deadline	8 November 2026, 23:59

Contents

A to F navigation and ID map ii

1 Part A: SRS and product scope 1
Software requirements specification . . . 1

2 Part B: Feature inventory and traceability 5
Feature inventory and traceability . . . 5

3 Part C: System-level design specification 8
System-level design specification . . . 8

4 Part D: Three-sprint plan 14
Three-sprint product plan . . . 14

5 Part E: Sprint 1 engineering contracts 16
Sprint 1 feature contracts and test specification . . . 16

6 Part F: working Sprint 1 and verification 26
6.1 Release record . . . 26
6.2 Local setup . . . 26
6.3 Synthetic finance demonstration . . . 26
6.4 Required student screenshots and screen dumps . . . 26
6.5 Recorded build, test, and migration results . . . 30
6.6 Acceptance-criteria to actual-test crosswalk . . . 30
6.7 Selected excerpts of actual automated tests . . . 32
6.8 Selected issue and PR evidence . . . 32
6.9 Contribution and AI-use record . . . 33
6.10 Limitations . . . 33

A to F navigation and ID map

Part	Pages	Contents / stable IDs
Part A: SRS and product scope	1–4	FR, BR, NFR, OQ, SRS-VAL, SRS-AC
Part B: Feature inventory	5–7	FEAT-01 to FEAT-14, requirement-to-feature-to-sprint matrix
Part C: System-level SDS	8–13	Architecture, data model, UI/API, roles, configuration, deployment, SDS-VAL
Part D: Three-sprint plan	14–15	Sprint goals, feature allocation, dependencies, gates, PLAN-VAL
Part E: Sprint 1 contracts	16–25	FEAT-01 to FEAT-08, F1 to F8 ACs, STS IDs, DoD
Part F: Working increment and evidence	26–33	Test results, AC-to-test mapping, issue and PR history, demo evidence, limitations

Page ranges come from PDF labels. Parts A to E summarize each required section and retain its stable IDs. Links open the full specifications, Issues, PRs, and source records.

1 Part A: SRS and product scope

Software requirements specification

Source: SRS, Issue #3.

Product and scope

Pockit is a persistent multi-User personal-finance system. Users manage only their own financial records. Administrators manage platform access and cannot view or modify financial data.

Scope	Included behavior
Product	Authentication, owned financial Accounts and Transactions, exact allocation, balances, summaries, category spending, history, favorites, and duplication.
Excluded	Period/shared budgets, recurring transactions, bank integration, real payment or messaging integrations, self-registration, password recovery, email verification, and advanced administration unless approved.

Actors

Actor	Responsibilities and permissions
User	Authenticates and manages only their own Accounts, Transactions, favorites, balances, and summaries. A User cannot access another User's financial records.
Administrator	Manages platform User accounts and access, including enabling/disabling Users. An Administrator cannot view, create, modify, or summarize any User's financial records.
Unauthenticated visitor	May use the login entry point only. All protected identity, User, and financial operations require authentication.

Functional requirements

ID	Requirement	Priority / allocation
FR-01	The system must authenticate a User and provide login, logout, and current-User identity with a documented session lifecycle.	Must; Sprint 1
FR-02	The system must enforce authentication, role, and record ownership on backend/API operations. A User must access only their own financial data; an Administrator must not access financial records or summaries.	Must; Sprint 1
FR-03	An Administrator must be able to manage platform User access, including enabling and disabling Users, without access to financial data.	Must; Sprint 1
FR-04	A User must be able to create and manage Accounts of the five types in BR-01.	Must; later sprint; Sprint 1 may use seeded Accounts
FR-05	A User must be able to establish an Account opening balance and record a Transaction date.	Must; Sprint 1 includes an opening bank balance and dated Transactions; broader management later
FR-06	A User must be able to record a payment/charge using a Cash or Loan Taken Account and allocate its total across the permitted destinations in BR-02.	Must; Sprint 1 includes a split credit-card purchase; remaining cases later
FR-07	A User must be able to record money received into a Cash Account and allocate its total across the permitted destinations in BR-03.	Must; Sprint 1 includes salary received; remaining cases later
FR-08	Every Transaction must have one or more allocation lines, and allocation amounts must equal the Transaction total. The system must reject an invalid allocation without partially applying the Transaction or its balance effects.	Must; Sprint 1
FR-09	The system must update all affected Account balances and category totals consistently with the operation; opening balances, transfers, borrowing, and loan repayments must not be classified as income or expenses.	Must; Sprint 1

Continued on next page

ID	Requirement	Priority / allocation
FR-10	A User must be able to view their Account balances, outstanding Loan Given/Loan Taken balances (including credit-card debt), income/expense summaries, category spending, and Transaction history.	Must; Sprint 1 includes bank balance, card debt, income/expense summary, category spending for the seeded categories, and history; broader summaries later
FR-11	A User must be able to mark an existing Transaction as a favorite and view favorite Transactions.	Must; later sprint
FR-12	A User must be able to duplicate an existing Transaction into a new editable Transaction; duplication must not alter the original.	Must; later sprint
FR-13	A User must be able to correct Transactions and search/filter Transaction history.	Must; later sprint; exact correction/audit behavior remains an open question
FR-14	The product must provide seeded synthetic demo data sufficient to demonstrate the Sprint 1 journey, including Users, a bank Account, a credit-card/Loan Taken Account, and Income/Expense categories.	Must; Sprint 1

Business rules

ID	Rule
BR-01	The supported Account types are exactly **Cash**, **Loan Given**, **Loan Taken**, **Income**, and **Expense**. Cash includes bank accounts and wallets; Loan Given represents money owed to a User; Loan Taken includes credit cards and other borrowed money. Income and Expense Accounts categorize the corresponding activity.
BR-02	For a payment/charge Transaction, the User selects a Cash or Loan Taken Account used. Allocation lines may represent Expense, repayment of Loan Taken, money lent to another party (Loan Given), or a transfer to another Cash Account.
BR-03	For a receive-money Transaction, the User selects the receiving Cash Account. Allocation lines may represent Income, repayment of Loan Given, new borrowing into Loan Taken, or a transfer from another Cash Account.
BR-04	A Transaction has at least one allocation line. The exact sum of its allocation lines must equal its total; a mismatch is rejected.
BR-05	Opening balances establish the starting balance for an Account. Subsequent Transactions change every affected balance according to the operation. A transfer changes the source and destination Cash balances by equal and opposite amounts.
BR-06	Cash increases on receipt and decreases on payment. Loan Taken outstanding debt increases when borrowed or charged and decreases when repaid. Loan Given outstanding receivable increases when money is lent and decreases when repaid. Income and Expense category totals increase only for their corresponding allocations. Opening balances and transfers do not create income or expenses.
BR-07	Borrowing and Loan Taken repayments, and lending and Loan Given repayments, affect the corresponding debt/receivable balances but are not classified as income or expenses.
BR-08	A User may read or change only their own financial records. An Administrator may manage platform User access but has no permission to view or modify financial records or summaries. The backend must enforce this boundary regardless of UI behavior.
BR-09	Favorites refer to existing Transactions. Duplicating a Transaction creates a distinct new editable Transaction and does not mutate the source Transaction.
BR-10	Monetary calculations must use exact decimal or integer-minor-unit arithmetic, never imprecise binary floating-point arithmetic. Sprint 1 uses THB at two fractional digits; inputs beyond that scale are rejected and values are never implicitly rounded. Additional currencies and mixed-currency behavior remain unresolved (OQ-01).
BR-11	Disabled Users cannot establish new sessions or access protected operations. Disabling a User immediately revokes all existing sessions; subsequent protected requests fail and new sign-ins are blocked (ADR-0002).

Non-functional requirements

ID	Requirement and verification measure
NFR-01 — Privacy and authorization	Every protected financial API operation must enforce authenticated User ownership and Administrator restrictions server-side. Verification must include unauthenticated access, cross-User access, forbidden role operations, and Administrator attempts to access financial records/summaries.

Continued on next page

ID	Requirement and verification measure
NFR-02 — Persistence	Accepted Accounts, Transactions, allocations, and balance effects must persist in the database and remain available after an application restart. Verify with a database-backed test or reproducible restart scenario.
NFR-03 — Financial integrity	Transaction totals, allocations, and all balance effects must be committed or rejected together; a rejected operation must leave no partial Transaction or balance change. Verify with valid, invalid, and boundary cases.
NFR-04 — Money precision	Monetary storage and arithmetic must avoid binary floating-point error. Sprint 1 uses exact THB minor units; tests cover 0.01 THB and reject excess precision without rounding. Future currency scales require explicit decisions.
NFR-05 — Secret and demo-data safety	Repository examples and tests use synthetic data. Real credentials, secrets, and personal/financial data must not be committed. Verify tracked examples/configuration and review changed files for sensitive data.
NFR-06 — Verifiability	Each implemented requirement must link to observable acceptance criteria and actual tests/evidence. Required authentication, authorization, persistence, and financial-invariant behavior must not be represented solely by UI mocks.
NFR-07 — Credential protection	If the application manages passwords, it must persist only suitable password hashes and never plaintext passwords. Verify the authentication configuration and persisted credential representation without exposing secret values.

Sprint 1 acceptance scope

Sprint 1 must demonstrate real login/logout/current identity and backend authorization; seeded Users, a bank Account, a Loan Taken credit-card Account, and Income/Expense categories; an opening bank balance; salary received into the bank; a credit-card purchase allocated across at least two Expense categories; and repayment from the bank Account. The User must see the resulting bank balance, credit-card debt, income/expense summary, category spending for the seeded categories, and Transaction history. The workflow must persist through the database and include meaningful business-rule rejection and tests.

Assumptions and boundaries

ID	Assumption or boundary
A-01	Theme 7 and approved project instructions define scope; the Sprint 1 example is a minimum vertical slice.
A-02	Seeded synthetic identities are sufficient; persistent PostgreSQL records and server-side authorization are required.
A-03	Two roles are defined: User and Administrator. Administrator financial-data exclusion is absolute.
A-04	Money/date decisions and unresolved questions remain recorded under OQ-01–OQ-07; later behavior is not inferred.

Open decisions

ID	Question / current resolution	Remaining decision or status
OQ-01	Sprint 1 uses one currency, THB, at two fractional digits, selected by Guntee under explicit product-owner delegation.	Teammate review is pending in Issue #8. Additional currencies, conversion, and mixed-currency behavior remain undecided and outside the Sprint 1 slice.
OQ-02	Exact arithmetic; reject amounts with more than two fractional digits; no implicit rounding of posted or derived Sprint 1 values.	Guntee reports this policy confirmed with the team/course. Any future calculation requiring rounding needs a separately recorded decision.
OQ-03	Require an explicit `YYYY-MM-DD` calendar date; store and return it without timezone conversion or an implicit current-date default. Sprint 1 summaries use all-time totals with no implicit date filter.	Guntee reports the date policy confirmed with the team/course. Any later date-filtering or alternate reporting-period behavior requires a separate decision.
OQ-04	What are the exact edit/delete/correction and audit-history rules for posted Transactions?	Before implementing Transaction correction.
OQ-05 — resolved	Guntee selected immediate revocation: disablement removes all stored sessions, blocks new session creation, and causes subsequent protected requests to fail.	Implement and test in Issue #8; teammate review is required in its PR. See ADR-0002.
OQ-06	What measurable performance, supported-browser, and accessibility targets are required for the course release?	Before release criteria are finalized.

Continued on next page

ID	Question / current resolution	Remaining decision or status
OQ-07	Sprint 1 monetary amounts and allocation lines must be greater than zero; reject a Loan Taken repayment above its outstanding debt and a Cash outflow above available balance.	Guntee reports these policies confirmed with the team/course. No product-specific maximum amount or allocation-line count is set; do not invent one. Any such limit remains open for a later approved scope.

SRS content checks

Check ID	SRS acceptance criterion	Validation method
SRS-AC-01 / SRS-VAL-01	Actors and responsibilities, including User and Administrator, are defined.	Review Section 2 against the approved brief and project constraints.
SRS-AC-02 / SRS-VAL-02	Functional, business, and important measurable non-functional requirements have stable identifiers.	Inspect FR/BR/NFR tables for unique, persistent identifiers and observable wording.
SRS-AC-03 / SRS-VAL-03	Scope, exclusions, assumptions, and resolved constraints are explicit.	Review Sections 1 and 7 against the brief; confirm excluded features are not included as requirements.
SRS-AC-04 / SRS-VAL-04	Account types and both transaction forms, allocation, and balance rules match the approved brief.	Cross-check BR-01 through BR-10 against Theme 7 text.
SRS-AC-05 / SRS-VAL-05	Sprint 1 is distinguished from later approved scope.	Cross-check Section 6 against the Theme 7 Sprint 1 example and “Later sprints” paragraphs, plus the Sprint 1 acceptance scope in Issue #8.
SRS-AC-06 / SRS-VAL-06	Unresolved questions are recorded instead of presented as settled decisions.	Review Section 8 and confirm unresolved decisions are not contradicted elsewhere.

2 Part B: Feature inventory and traceability

Feature inventory and traceability

Source: Feature inventory, Issue #4.

The SRS defines 32 requirement IDs: 14 FRs, 11 BRs, and 7 NFRs. Every ID maps to a feature. Sprint 2 and 3 allocations remain proposed.

Feature inventory

Feature ID	Feature / user outcome	Planned sprint	Dependencies / gates
FEAT-01	**Authentication and session lifecycle:** Users can log in, log out, and retrieve current identity using the documented session lifecycle.	Sprint 1 — required	Project Setup #9 is complete. Disabled-user revocation follows ADR-0002 and is implemented/ tested under Issue #8.
FEAT-02	**Server-side role and ownership controls:** Users access only their own financial data; Administrators manage platform access without accessing any financial record or summary.	Sprint 1 — required	FEAT-01. Administrator finance-data denial is tested at the backend/API boundary.
FEAT-03	**Synthetic Sprint 1 demo data:** Seed demo Users, a bank/Cash Account, a credit-card/Loan Taken Account, and Income/Expense categories.	Sprint 1 — required	Project Setup #9; use synthetic data only.
FEAT-04	**Opening balance and transaction date foundation:** Establish the opening bank balance and record dates for the Sprint 1 Transactions.	Sprint 1 — required	FEAT-03; currency/date open questions must remain visible and must not be guessed.
FEAT-05	**Receive-money vertical slice:** Record salary into the bank/Cash Account with an Income allocation and the required balance effects.	Sprint 1 — required	FEAT-01–FEAT-04.
FEAT-06	**Payment/charge vertical slice:** Record a credit-card purchase split across at least two Expense categories and record repayment from the bank Account.	Sprint 1 — required	FEAT-01–FEAT-05; allocation and balance effects must be atomic.
FEAT-07	**Sprint 1 balances and views:** Show the resulting bank balance, outstanding card debt, income/expense summary, category spending for seeded categories, and Transaction history.	Sprint 1 — required	FEAT-04–FEAT-06; values must come from persisted domain data.
FEAT-08	**Cross-cutting quality and evidence:** Apply server authorization, persistence, atomic financial integrity, exact money handling, synthetic-data/ secret safety, credential protection, and requirement-linked verification across all features.	Sprints 1–3 — continuous	Applies to every feature; detailed acceptance tests belong in each Feature-Level SDS/STS.
FEAT-09	**Full Account lifecycle:** Users create and manage all five Account types beyond the seeded Sprint 1 setup.	Sprint 2 — proposed	FEAT-01, FEAT-02, FEAT-03; confirm permitted Account operations in Issue #6/feature contracts.
FEAT-10	**Expanded money movement:** Support the remaining approved payment/charge and receive-money allocations, including Loan Given/ Loan Taken activity and Cash transfers.	Sprint 2 — proposed	FEAT-09; OQ-01, OQ-02, and OQ-07 must be resolved before final amount validation.
FEAT-11	**Transaction history search and filtering:** Find and filter a User's persisted Transaction history.	Sprint 2 — proposed	FEAT-07; preserve User ownership restrictions.
FEAT-12	**Posted Transaction correction:** Correct a Transaction while preserving consistent balances and required history/audit behavior.	Sprint 2 — proposed; gated	FEAT-05, FEAT-06, FEAT-10; OQ-04 must be resolved before the correction contract is approved.
FEAT-13	**Favorites and duplication:** Mark/view favorite Transactions and duplicate an existing Transaction into a separate editable Transaction.	Sprint 3 — proposed	FEAT-07; duplication must not modify the original.
FEAT-14	**Expanded category analysis and reports:** Extend beyond the Sprint 1 seeded-category spending view to the approved later-sprint summaries/reporting scope.	Sprint 3 — proposed; details open	FEAT-07, FEAT-10; report types, periods, and filters need clarification before a feature contract.

Requirement-to-feature-to-sprint matrix

Requirement ID	Supporting feature(s)	Sprint coverage
FR-01	FEAT-01	Sprint 1
FR-02	FEAT-01, FEAT-02, FEAT-08	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
FR-03	FEAT-02	Sprint 1
FR-04	FEAT-09	Sprint 2 proposed
FR-05	FEAT-04	Sprint 1
FR-06	FEAT-06, FEAT-10	Sprint 1, Sprint 2 proposed
FR-07	FEAT-05, FEAT-10	Sprint 1, Sprint 2 proposed
FR-08	FEAT-05, FEAT-06, FEAT-10	Sprint 1, Sprint 2 proposed
FR-09	FEAT-05, FEAT-06, FEAT-07, FEAT-10, FEAT-14	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
FR-10	FEAT-07, FEAT-11, FEAT-14	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
FR-11	FEAT-13	Sprint 3 proposed
FR-12	FEAT-13	Sprint 3 proposed
FR-13	FEAT-11, FEAT-12	Sprint 2 proposed
FR-14	FEAT-03	Sprint 1
BR-01	FEAT-03, FEAT-09	Sprint 1, Sprint 2 proposed
BR-02	FEAT-06, FEAT-10	Sprint 1, Sprint 2 proposed
BR-03	FEAT-05, FEAT-10	Sprint 1, Sprint 2 proposed
BR-04	FEAT-05, FEAT-06, FEAT-08, FEAT-10, FEAT-12	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
BR-05	FEAT-04, FEAT-07, FEAT-09, FEAT-10	Sprint 1, Sprint 2 proposed
BR-06	FEAT-04, FEAT-05, FEAT-06, FEAT-07, FEAT-10, FEAT-14	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
BR-07	FEAT-05, FEAT-06, FEAT-07, FEAT-10, FEAT-14	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
BR-08	FEAT-02, FEAT-08, FEAT-11, FEAT-12	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
BR-09	FEAT-13	Sprint 3 proposed
BR-10	FEAT-04, FEAT-05, FEAT-06, FEAT-08, FEAT-10, FEAT-12	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
BR-11	FEAT-01, FEAT-02, FEAT-08	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
NFR-01	FEAT-02, FEAT-08, FEAT-11, FEAT-12	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
NFR-02	FEAT-03, FEAT-04, FEAT-05, FEAT-06, FEAT-07, FEAT-08, FEAT-09, FEAT-10, FEAT-11, FEAT-12, FEAT-13, FEAT-14	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
NFR-03	FEAT-05, FEAT-06, FEAT-08, FEAT-10, FEAT-12	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
NFR-04	FEAT-04, FEAT-05, FEAT-06, FEAT-08, FEAT-10	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
NFR-05	FEAT-03, FEAT-04, FEAT-08, FEAT-09	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
NFR-06	FEAT-07, FEAT-08, FEAT-11, FEAT-12, FEAT-13, FEAT-14	Sprint 1, Sprint 2 proposed, Sprint 3 proposed
NFR-07	FEAT-01, FEAT-02, FEAT-08	Sprint 1, Sprint 2 proposed, Sprint 3 proposed

Coverage gaps and decisions

Open item	Treatment / follow-up
Sprint 2/3 allocation	Proposed only; confirm in team sprint planning.
Reports	FEAT-14 is provisional; confirm whether FR-10 defines report scope before contracting it.
Transaction correction	FEAT-12 is gated by OQ-04; decide correction and audit behavior first.
Currency, date, limits	Apply recorded Sprint 1 policies only; future currency/period/limit behavior remains open.

Inventory validation

Check ID	Validation	Result
INV-VAL-01	Compare feature-table and matrix mappings in both directions; confirm every SRS FR, BR, and NFR appears exactly once in the matrix.	**PASS:** all 32 SRS requirement IDs have matrix rows, and every feature-to-requirement link matches the reverse matrix link.
INV-VAL-02	Confirm each feature has a planned sprint and supporting SRS IDs, and each matrix coverage label equals the union of its mapped features' planned sprints.	**PASS:** all 14 features have a sprint and requirements; all 32 matrix coverage labels match their mapped feature sprints, including FEAT-08's continuous coverage.
INV-VAL-03	Cross-check five account types, transaction allocation forms, Sprint 1 slice, and later scope against the approved Theme 7 brief and Issue #8.	**PASS:** manual cross-check against Theme 7 and Issue #8; the report-detail gap is explicitly recorded.
INV-VAL-04	Review duplicate mappings, gaps, and provisional S2/S3 allocations; confirm every unresolved decision has an owner/follow-up.	**PASS:** repeated mappings are intentional; report detail and proposed S2/S3 allocations have explicit follow-ups.
INV-VAL-05	Check terminology and stable identifiers against the SRS.	**PASS:** matrix IDs and domain terms match the SRS; no duplicate requirement or feature identifiers.

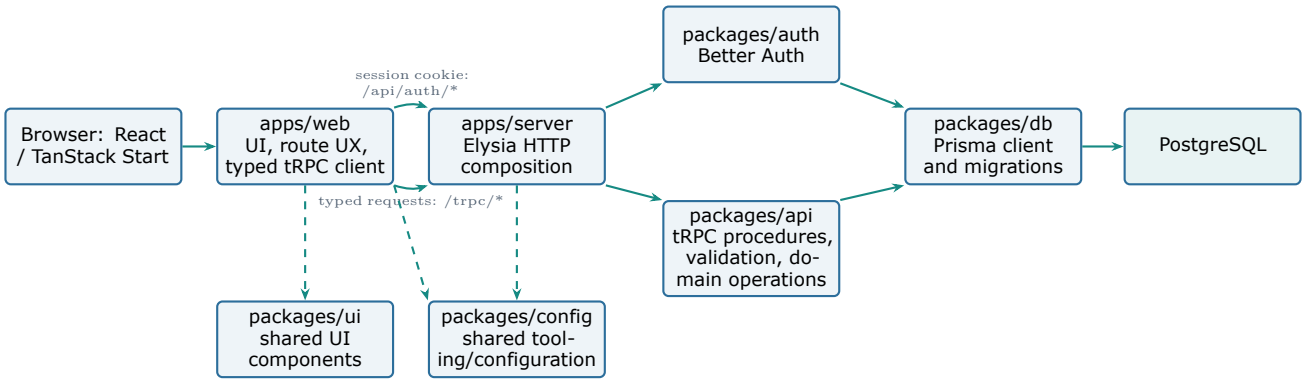
3 Part C: System-level design specification

System-level design specification

Source: System-Level SDS, Issue #5, ADR-0001, ADR-0002.

Architecture and boundaries

Area	Present in the repository	Planned by the approved product design
Web	React 19, TanStack Start/Router, TanStack Query, typed tRPC client, Better Auth client, Tailwind/ shadcn UI. The protected route currently performs a client-side session check.	Login and authenticated finance workflows, with UI route guards as a usability layer only.
HTTP/API server	Elysia exposes Better Auth at `/api/auth/*`, tRPC at `/trpc/*`, and a health endpoint.	Financial and administration procedures with server-side role and owner checks.
API package	tRPC router and a `protectedProcedure` that currently verifies only that a session exists.	Domain input validation, authorization boundaries, financial operations, and summaries. A session alone is not sufficient authorization.
Authentication	Better Auth email/password configuration with a Prisma/PostgreSQL adapter and database-backed sessions.	Seeded demo identities, role and enabled-status enforcement, and documented session lifecycle.
Database	Prisma 7 with PostgreSQL and versioned migrations; current schema contains Better Auth identity/session models only.	User-owned financial Accounts, Transactions, allocation lines, and any required role/status fields.
Deployment	Dockerfiles and Compose files support local web/ server containers and a separate persistent PostgreSQL development service.	Reproducible Docker delivery and a later GCP deployment, scheduled for Lab 5 rather than a Sprint 1 prerequisite.



Dashed arrows show shared-package usage.

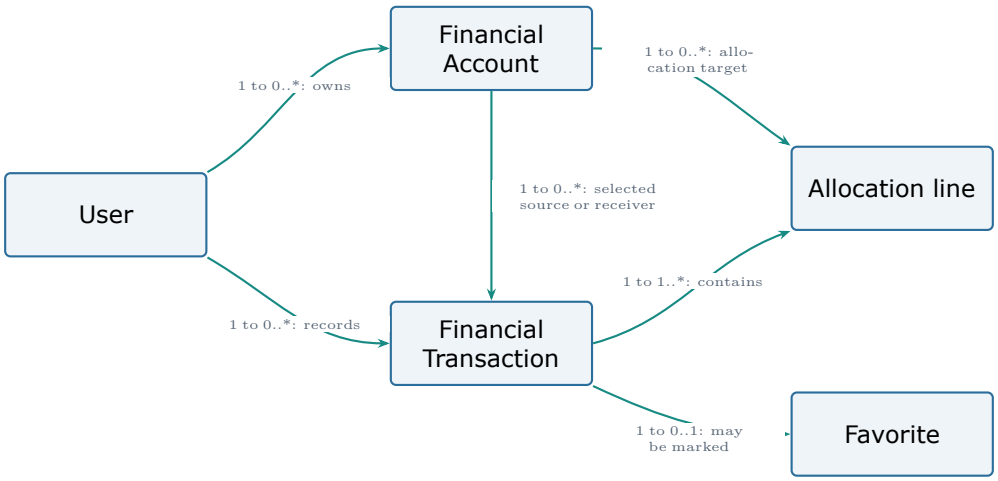
Component	Responsibility and boundary
`apps/web`	Render the User and Administrator experiences; use Better Auth for authentication and the typed tRPC client for application operations. UI hiding and route guards improve navigation but do not grant or deny data access. Browser money values cross the API as decimal strings, never JavaScript floating-point values.
`apps/server`	Compose the Elysia HTTP application, CORS policy, Better Auth routes, tRPC fetch handler, and request context. The context resolves the Better Auth session from request headers and makes the database available to API procedures. It is the HTTP boundary, not the owner of finance rules.
`packages/auth`	Configure Better Auth and its Prisma adapter; manage credential authentication and persistent sessions. Identity, role, and enabled status must be available to authorization checks. Passwords must be stored only as suitable hashes when managed by the application.
`packages/api`	Define the typed tRPC API, input schemas, authorization procedures, and application/ domain operations. Keep business invariants server-side. Procedures must resolve the User from the session and constrain every finance query/mutation to that User.
`packages/db`	Own Prisma client setup, database schema, and versioned migrations. Persist Better Auth and financial data in PostgreSQL. Database constraints and transactions backstop API validation.

Continued on next page

Component	Responsibility and boundary
`packages/ui`	Provide shared presentation components; no domain authorization or financial persistence.
`packages/config`	Share tooling and configuration conventions; do not put secrets or runtime financial state here.

Data model

Entity	Key information	Relationships and invariants
User	Better Auth identity plus application role (`User` or `Administrator`) and enabled/disabled status.	One User owns many Financial Accounts and Transactions. Administrators manage platform access but are never granted finance access. Disabling a User immediately revokes all stored sessions, prevents new sessions, and denies subsequent protected requests (ADR-0002).
Financial Account	Owner User, display name, one of the five Account types, currency, and opening-balance information where applicable. The finance entity should be named `FinancialAccount` (or an equally unambiguous name), not Better Auth's existing Prisma `Account` model.	Belongs to exactly one User. A User owns zero or more. Account type constrains where it can be used as a source, receiver, or allocation destination.
Financial Transaction	Owner User, form (`payment/charge` or `receive money`), total amount, transaction date, selected source/receiving Account, and optional description.	Belongs to exactly one User; has one or more allocation lines. The selected Account and every allocation Account must belong to the same owner.
Allocation line	Parent Transaction, destination/counterpart Account, exact amount, and an operation/ direction where needed to distinguish transfer, borrowing, or repayment.	Belongs to one Transaction and references a Financial Account. At least one line is required; the exact sum equals the parent total.
Favorite association	Existing Transaction marked as a favorite by its owner.	References an existing Transaction and its owning User; cannot expose another User's Transaction. A field or separate relation is an implementation detail for the feature contract.



Cardinalities run from the source entity to the related entity.

Type	Meaning and balance interpretation
Cash	Bank account or wallet. Its balance increases on receipt and decreases on payment; a Cash-to-Cash transfer changes both Accounts by equal and opposite amounts.
Loan Given	Money owed to the User. Lending increases the receivable; repayment decreases it. It is not Income.
Loan Taken	Credit-card or other borrowed-money debt. Borrowing or a charge increases outstanding debt; repayment decreases it. It is not Expense merely because it is repaid.
Income	Category for income activity. Its total increases only from an Income allocation, not from a transfer, borrowing, opening balance, or loan repayment.

Continued on next page

Type	Meaning and balance interpretation
Expense	Category for spending activity. Its total increases only from an Expense allocation, not from a transfer, borrowing, opening balance, or loan repayment.

Transactions and balances

Form	Selected Account	Allocation destinations
Payment or charge	Cash used for payment, or Loan Taken used for a charge.	Expense; repayment of Loan Taken; lending to Loan Given; or transfer to another Cash Account.
Receive money	Cash receiving the funds.	Income; repayment of Loan Given; new borrowing into Loan Taken; or transfer from another Cash Account.

The ledger in PostgreSQL is authoritative. Create each Transaction and all allocation lines atomically. Account and category values derive from opening positions plus committed postings.

Event	Effect
Opening position	Establish the starting balance of a balance-bearing Account; it is not Income or Expense. The account type determines whether the natural balance represents Cash, receivable, or debt.
Payment/charge selected from Cash	Decrease the selected Cash balance by the Transaction total.
Payment/charge selected from Loan Taken	Increase that Loan Taken Account's outstanding debt by the Transaction total.
Receive-money Transaction	Increase the selected receiving Cash Account by the Transaction total.
Expense allocation	Increase the Expense category total by that allocation amount.
Income allocation	Increase the Income category total by that allocation amount.
Loan Taken repayment allocation	Decrease the referenced Loan Taken outstanding debt. It does not create Income or Expense.
New Loan Taken borrowing allocation	Increase the referenced Loan Taken outstanding debt. It does not create Income or Expense.
Loan Given lending allocation	Increase the referenced Loan Given receivable. It does not create Income or Expense.
Loan Given repayment allocation	Decrease the referenced Loan Given receivable. It does not create Income or Expense.
Cash transfer allocation	Increase the destination Cash balance for an outgoing transfer, or decrease the source Cash balance for an incoming transfer. The counterpart effect must be equal and opposite; neither side is Income or Expense.

UI, API, and money representation

Area	Boundary
User UI	Login, owned Accounts/Transactions, dashboard, summaries, category spending, history; later favorites and duplication.
Administrator UI	Platform User access only; no finance screens or queries.
Money transport	Base-10 decimal strings; exact integer-minor-unit/decimal arithmetic and PostgreSQL NUMERIC/Prisma Decimal; never JavaScript floating point.

API area	Intended operations	Authorization boundary
Authentication	Better Auth login, logout, and current session/identity.	Public login; session required for identity and protected app operations.
Accounts	Read and manage the current User's Financial Accounts, opening balances, and balances.	User-owned finance operations only; Administrator denied.
Transactions	Create/read history and details; later correction and duplication.	Session + User role + owner checks on Transaction and every referenced Account.
Summaries	Account balances, debt/receivables, income/expense totals, category spending.	User role only; query scope always derived from session identity.
Favorites	Mark/unmark and list the current User's favorite Transactions.	User role and Transaction ownership; Administrator denied.
Administrator User access	List/manage platform User access and enabled status.	Administrator only; response fields must not include finance records or summaries.

Roles and privacy

Operation	Unauthenticated visitor	User	Administrator
Login	Allowed	Allowed	Allowed
Logout/current identity	No session / rejected	Own session only	Own session only
Read or change own financial Accounts/Transactions/favorites	Rejected	Allowed, own records only	**Forbidden**
Read another User's financial data	**Forbidden**	**Forbidden**	**Forbidden**
Read financial summaries or category spending	**Forbidden**	Allowed, own records only	**Forbidden**
Manage platform User access and enabled status	**Forbidden**	**Forbidden**	Allowed, excluding financial data

Every finance query uses the session-derived owner. Deny Administrator finance access at API boundaries, including direct IDs and summary endpoints.

Configuration and sessions

Configuration	Consumer	Handling
`DATABASE_URL`	Server and Prisma	Secret; use local ignored environment file in development and the deployment secret store in production.
`BETTER_AUTH_SECRET`	Server/Better Auth	Secret; minimum 32 characters per current schema. Do not put in source, Docker build args, browser configuration, or logs.
`BETTER_AUTH_URL`	Server/Better Auth	Server runtime URL; environment-specific.
`CORS_ORIGIN`	Server	Explicit trusted browser origin; avoid permissive wildcard configuration with credentialed requests.
`VITE_SERVER_URL`	Web	Public server endpoint; it is a client-visible URL, not a secret.
`SERVER_URL`	Web server-side rendering	Internal/server-side API URL where applicable; do not expose internal credentials through it.
`NODE_ENV`	Apps/packages	Environment mode validated through schemas.

Better Auth stores sessions in PostgreSQL. Logout invalidates the session. Disabled Users cannot create sessions; disablement deletes stored sessions, and protected requests recheck current User status. See ADR-0002.

Deployment

Environment	Design
Local	Compose web/server containers and PostgreSQL 17 with a persistent volume; apply versioned migrations.
Lab 5 target	Artifact Registry images; Cloud Run web/API; Cloud SQL PostgreSQL; Secret Manager; Cloud Logging; GitHub Actions with Workload Identity Federation.
Status	Docker/GCP deployment is planned for Lab 5, not a Sprint 1 cloud prerequisite.

Money decisions

Rule / decision	Sprint 1 behavior
OQ-01 Currency	THB only at two fractional digits; other currencies and conversion remain open.
OQ-02 Precision	Exact decimal strings/minor units; excess precision is rejected; no implicit rounding. See ADR-0001.
OQ-03 Date	Explicit YYYY-MM-DD; preserve without timezone conversion; summaries are all-time.
BR-04 Allocation	One or more lines; exact allocation sum equals transaction total.
BR-05–BR-07 Balances	Opening positions plus committed postings; transfers, borrowing, and repayments are not income/expense.
OQ-07 Amounts	Positive amounts; reject repayment above debt and Cash outflow above available balance; no invented maximum.

OQ-04 correction/audit behavior and OQ-06 measurable quality targets remain open. See Part A for all recorded OQ statuses.

Requirement-to-design map

SRS requirements	Design support
FR-01, FR-02, FR-03; BR-08, BR-11; NFR-01, NFR-07	Better Auth identity/session boundary, User role/status, role/ownership matrix, server-side authorization; Admin finance denial.

Continued on next page

SRS requirements	Design support
FR-04, FR-05, FR-14; BR-01, BR-05	Financial Account model, five Account types, opening position, seeded synthetic data, PostgreSQL persistence.
FR-06, FR-07, FR-08; BR-02, BR-03, BR-04	Two Transaction forms, typed allocation lines, exact total validation, account type and owner validation, atomic writes.
FR-09, FR-10; BR-05, BR-06, BR-07; NFR-02, NFR-03, NFR-04	Ledger-derived balances and summaries, exact monetary representation, persistent data, no partial effects, no misclassification of transfer/debt activity.
FR-11, FR-12; BR-09	Owner-scoped favorite association and distinct editable duplication behavior in later feature contracts.
FR-13	Transaction correction/history design deferred until SRS OQ-04 is resolved; history query remains owner-scoped.
NFR-05, NFR-06	Varlock/environment-secret policy, synthetic data, test/evidence requirements, versioned migrations.

Verification seams and content checks

Seam	What it verifies
API integration	Observable results, authorization, validation, and persisted state.
PostgreSQL integration	Persistence, exact values, atomicity, and derived balances.
HTTP/Auth	Session transport and protected-request behavior.
UI workflow	Accessible visible behavior; never a substitute for API/DB evidence.

Validation ID	Check	Expected evidence
SDS-VAL-01	Architecture and stack agree with the current repository and distinguish existing capabilities from planned product work.	Review Sections 1–2 against package manifests and application boundaries.
SDS-VAL-02	Logical data model covers all five Account types, Transaction forms, allocation lines, owner relationships, and favorites.	Review Sections 3–4 against SRS FR/BR scope.
SDS-VAL-03	UI/API boundaries and business interactions preserve the SRS forms, allocations, and atomicity.	Review Sections 4–5 against FR-06..FR-10 and BR-02..BR-07.
SDS-VAL-04	User and Administrator permissions explicitly deny Administrator access to finance data and summaries.	Review Section 6; confirm every finance operation has a server-side role/owner boundary.
SDS-VAL-05	Configuration and secret handling are documented without exposing credentials.	Review Section 7 against Varlock schemas and inspect repository changes for secret values.
SDS-VAL-06	Docker/GCP deployment is persistent and correctly marked as planned Lab 5 work, not Sprint 1 cloud delivery.	Review Section 8 against Compose files and the Lab 5 deployment guide.
SDS-VAL-07	Precision, rounding, date, allocation equality, and balance calculations are explicit, with remaining currency/limit questions clearly separated from Sprint 1 decisions.	Review Section 9 and ADR-0001 against SRS BR-10 and the current OQ-01..OQ-03/OQ-07 status.

Decision ownership

SRS question	Design implication / resolution point
OQ-01 — currency and precision	THB with two fractional digits is selected for Sprint 1 by Guntee under explicit delegation; teammate review is pending in Issue #8. Other currencies, conversion, and mixed-currency behavior remain open.
OQ-02 — rounding and excess precision	Sprint 1 rejects values beyond two fractional digits and does not implicitly round; Guntee reports this decision confirmed with the team/course. Future rounding needs a separate rule.
OQ-03 — date/time zone and default Transaction date	Sprint 1 requires explicit `YYYY-MM-DD`, no timezone conversion/default, and all-time summaries; Guntee reports confirmation with the team/course. Future filters need a separate contract.
OQ-04 — correction and audit	Decide edit/delete/correction and historical audit behavior before implementing correction.
OQ-05 — disabled-session revocation	Resolved by ADR-0002: immediate persisted-session deletion, new-session rejection, and protected-request denial. Implementation and teammate review remain in Issue #8.
OQ-06 — performance/browser/accessibility targets	Set measurable release targets before release acceptance.

Continued on next page

SRS question	Design implication / resolution point
OQ-07 — amount limits and invalid values	Sprint 1 rejects non-positive amounts, repayment above debt, and Cash outflow above balance; no product-specific maximum or allocation-line count is defined. Guntee reports team/course confirmation. Future limits remain open.

4 Part D: Three-sprint plan

Three-sprint product plan

Source: Three-sprint plan, Issue #6.

Later-sprint allocations are proposals, not fixed commitments. Actual sprint dates and capacity are set by the team; no estimates are invented.

Sprint goals and exit outcomes

Sprint	Goal and exit outcome	Feature allocation	Course checkpoint
Sprint 1 — required baseline	Release a small, persistent product journey: a User records an opening bank balance, receives salary, splits a credit-card purchase across Expense categories, repays the card, and sees correct balances/history/summaries. Authentication, backend authorization, meaningful rejection, and tests are part of the journey.	FEAT-01–FEAT-07; FEAT-08 applies continuously.	Project Deliverable 1: **8 November 2026**. Sprint 1 deployment/repeatable delivery for Lab 5: **22 November 2026**.
Sprint 2 — complete core account and money movement	Expand from seeded accounts and the minimum journey to the remaining approved account lifecycle, transaction allocation, and history operations. Preserve exact arithmetic, owner isolation, and atomic persistence.	FEAT-09–FEAT-12; FEAT-08 applies continuously. FEAT-12 is gated by OQ-04.	Contribute to a substantially complete product and Project Deliverable 2 by **29 November 2026**.
Sprint 3 — complete remaining user outcomes and acceptance	Deliver private favorites and editable duplication, complete only the report/category-analysis scope approved by the team/course, and finish whole-product regression, UAT, and release readiness.	FEAT-13–FEAT-14; FEAT-08 applies continuously. FEAT-14's report detail is gated by scope clarification.	Product substantially complete by **29 November 2026**; final fixes, verification and submission preparation lead to Final Delivery on **13 December 2026**.

Dependencies and decision gates

Order	Feature IDs	Dependency / gate
Sprint 1	FEAT-01/02 → FEAT-03 → FEAT-04 → FEAT-05/06 → FEAT-07	FEAT-08 applies throughout; complete the authenticated, authorized, persistent minimum journey.
Sprint 2	FEAT-09 → FEAT-10 → FEAT-11	FEAT-12 requires approved OQ-04 correction/audit behavior.
Sprint 3	FEAT-13; FEAT-14	Confirm report scope for FEAT-14 before its feature contract.

Course checkpoints

Checkpoint	Required outcome
8 Nov 2026	Deliverable 1: baseline and Sprint 1 working journey.
22 Nov 2026	Lab 5: deployed Sprint 1 and repeatable delivery.
29 Nov 2026	Deliverable 2; product substantially complete.
13 Dec 2026	Final product, report, and demo video.

Verification and release gates

Sprint	Required verification before calling the increment accepted
Sprint 1	Complete Issue #8's opening balance → salary → split card purchase → repayment journey. Use real persistence. Verify valid/invalid login, unauthenticated access, cross-User access, forbidden role operations, Administrator access management (including enabling/disabling Users) and finance-data denial, exact allocation equality, meaningful invalid-transaction rejection with no partial effects, correct balances/summaries/history, and synthetic seed data. Link every acceptance ID to actual test IDs/results.
Sprint 2	Database/API tests for all five Account types and allowed operations; both transaction forms and remaining allocations; equal/opposite Cash transfers; loan balances not misclassified as Income/Expense; exact arithmetic and atomic rejection; owner-scoped search/filter; and correction only if OQ-04 is approved. Include negative, boundary, and cross-User cases.

Continued on next page

Sprint	Required verification before calling the increment accepted
Sprint 3	Verify favorite ownership, duplicate independence/editability, approved reporting/category outcomes, and full regression of previous sprints. Complete teammate-reviewed UAT/re-tests, deployment/restart persistence checks, defect status, and final release evidence.

Plan validation

Check ID	Acceptance evidence	Result
PLAN-VAL-01	Every feature FEAT-01–FEAT-14 is allocated to Sprint 1, 2, or 3 (with FEAT-08 explicitly continuous), with supporting SRS IDs linked to the feature inventory.	**PASS:** automated check confirmed all 14 sprint assignments and expanded FR/BR/NFR mappings match the feature inventory.
PLAN-VAL-02	Dependencies and OQ gates identify the required sequence; gated work is not silently treated as approved or complete.	**PASS:** manual review of Sections 3 and 6 against the SRS/SDS confirms OQ-01/02/03/04/05/06/07 gates and required sequencing.
PLAN-VAL-03	Sprint 1 preserves the minimum authenticated, authorized, persistent journey and meaningful business-rule rejection/test evidence from the brief and Issue #8.	**PASS:** cross-checked against Theme 7 and Issue #8; includes authentication, server authorization, persistence, rejection, and tests.
PLAN-VAL-04	All remaining approved later-sprint scope is assigned or its requirements gap/gate is explicitly recorded; no feature is silently dropped.	**PASS:** all 14 features are allocated; correction and report-scope gates remain explicit.
PLAN-VAL-05	Course dates, examination constraints, two-student participation, and unknown capacity are considered without inventing sprint dates, hours, or velocity.	**PASS:** course PDF milestones/exam windows cross-checked; team capacity is recorded as two students with no invented velocity or sprint dates.
PLAN-VAL-06	Goals and allocations agree with the SRS and System-Level SDS; proposed later-sprint assignments agree with the feature inventory.	**PASS:** manual cross-check against SRS, SDS, and inventory; all feature allocations and supporting IDs match.

5 Part E: Sprint 1 engineering contracts

Sprint 1 feature contracts and test specification

Sources: Feature contracts, STS, Issue #7, Issue #8.

This section summarizes FEAT-01–FEAT-08. It retains every acceptance and test ID; implementation history and actual results are in Part F and the linked Issues/PRs.

Shared permission contract

Actor	Allowed in this contract	Not allowed
Unauthenticated visitor	Submit login credentials.	Protected identity, User-management, or financial operations.
User	Read and mutate only their own Sprint 1 Accounts and Transactions; see only their own balances, summaries, categories, and history.	Another User's records or Administrator-only access management.
Administrator	Read the minimum platform identity/access fields needed to enable or disable Users.	Any financial Account, Transaction, allocation, favorite, balance, category total, summary, or financial history operation.

Shared data and API rules

Contract	Required behavior
Identity	Derive User, role, and enabled status from the validated server session; ignore client-supplied owner/role authority.
Data	PostgreSQL is the source of truth. Transactions and allocations persist together or not at all.
Money	Use decimal strings across JSON and exact arithmetic/storage; no binary floating-point. Apply recorded Sprint 1 precision and date policies.
Ownership	Check source, receiver, allocation, filter, and direct-record IDs against the session owner.
Evidence	For rejected writes, verify no transaction, allocation, balance, category, summary, or history change.

Open feature decisions

Decision	Contract treatment	Gate
OQ-01 — currency codes and scale	Sprint 1 uses THB only at two fractional digits, selected by Guntee under explicit delegation; money is represented exactly. See ADR-0001.	Teammate review is requested in the Issue #8 PR. Other currencies, conversion, and mixed-currency behavior remain out of scope/ open.
OQ-02 — excess precision and rounding	Reject more than two fractional digits; do not implicitly round posted or derived Sprint 1 values. Guntee reports team/ course confirmation; see ADR-0001.	No Sprint 1 gate remains. Future calculations needing rounding require a separately approved rule.
OQ-03 — date/ time-zone and summary period	Require explicit `YYYY-MM-DD`, store/return unchanged, no timezone conversion/default; Sprint 1 summaries are all-time. Guntee reports team/course confirmation; see ADR-0001.	No Sprint 1 gate remains. Date filters and different reporting periods need a later contract.
OQ-05 — disabled User with an existing session (resolved)	Guntee selected immediate revocation, documented in ADR-0002.	Disablement removes all stored sessions atomically with status update; disabled Users cannot create new sessions; later protected requests fail. Implement and verify in Issue #8; teammate review is required in its PR.
OQ-06 — quality targets	Follow the existing NFRs and verify the required functional/security behaviors.	Performance, supported-browser, and accessibility thresholds remain open; do not invent release targets in this contract.
OQ-07 — amount constraints	Sprint 1 amounts and allocation lines must be positive; reject card repayments above debt and Cash outflows above available balance. Guntee reports team/course confirmation; see ADR-0001.	No product-specific maximum amount or allocation-line count is set. Do not invent either; future limits remain open.

Feature-level contracts

FEAT-01 — Authentication and session lifecycle

Contract	Summary
Outcome	Seeded Users and Administrators sign in, resolve their own identity, and log out. Public registration and recovery are excluded.
UI/API/data	Use Better Auth login and PostgreSQL sessions. Identity comes from the server session; passwords are hashed and cookies follow trusted-origin/security settings.
Dependencies/gates	Project Setup #9; Better Auth/Prisma; ADR-0002 session revocation.
DoD/evidence	STS-AUTH: verify valid/invalid login, identity, logout, disabled-user revocation, and credential safety over HTTP/PostgreSQL.

FEAT-02 — Server-side role and ownership controls

Contract	Summary
Outcome	Users access only their own finance records. Administrators manage platform access but never finance data.
UI/API/data	Enforce role and ownership in tRPC/API operations for direct IDs, filters, nested Accounts, and summaries. Admin responses contain identity/access fields only.
Dependencies/gates	FEAT-01; ADR-0002; session and owner checks are server-authoritative.
DoD/evidence	STS-AUTHZ: verify owned access, cross-User denial, unauthenticated denial, Admin operations, and Admin finance denial at the API boundary.

FEAT-03 — Synthetic Sprint 1 demo data

Contract	Summary
Outcome	Provide synthetic User/Admin identities, bank/Cash and card/Loan Taken Accounts, Income, and at least two Expense categories.
UI/API/data	Seed through the normal auth and finance paths. Records have explicit owners; reruns are additive/idempotent and do not reset unrelated data.
Dependencies/gates	Project Setup #9; FEAT-01/02; ADR-0001/0002.
DoD/evidence	STS-SEED: verify a fresh seed, owner isolation, safe repeat runs, and synthetic-only fixtures.

FEAT-04 — Opening balance and explicit Transaction date foundation

Contract	Summary
Outcome	Record the opening Cash position and explicit dates for Sprint 1 Transactions.
UI/API/data	Persist an exact THB opening balance on the seeded Cash Account. Require YYYY-MM-DD dates; do not turn the opening position into Income.
Dependencies/gates	FEAT-03; ADR-0001; posted-baseline correction remains gated by OQ-04.
DoD/evidence	STS-OPEN: verify persistence, unchanged date round-trip, precision, ownership, Account type, and positive-amount rejection.

FEAT-05 — Salary receipt into Cash

Contract	Summary
Outcome	Record salary received into Cash with an Income allocation and correct persisted effects.
UI/API/data	Submit receiving Cash Account, date, total, and one or more Income lines as decimal strings; reject mismatches atomically.
Dependencies/gates	FEAT-01–FEAT-04; ADR-0001/0002.
DoD/evidence	STS-RECEIVE: verify salary, exact balance/category effects, persistence, and invalid-write rollback.

FEAT-06 — Credit-card purchase and bank repayment

Contract	Summary
Outcome	Record a Loan Taken card purchase allocated to Expense categories, then repay it from Cash. The demo splits one purchase across two categories.
UI/API/data	Persist explicit-date payment/charge and repayment Transactions. Enforce account types, exact totals, positive amounts, available Cash, and outstanding debt.

Continued on next page

Contract	Summary
Dependencies/gates	FEAT-01–FEAT-05; ADR-0001/0002.
DoD/evidence	STS-CARD: verify split and one-line purchases, repayment, balances, category totals, and unchanged state after rejection.

FEAT-07 — Persisted balances, summary, category spending, and history

Contract	Summary
Outcome	Show persisted Cash/debt balances, Income/Expense totals, seeded-category spending, and Transaction history.
UI/API/data	Use owner-scoped API reads from PostgreSQL. Sprint 1 summaries are all-time; opening balances, borrowing, and repayments are not Income/Expense.
Dependencies/gates	FEAT-04–FEAT-06; ADR-0001/0002.
DoD/evidence	STS-VIEW: compare summary/history with persisted records and verify a fresh-client read.

FEAT-08 — Cross-cutting integrity, security, and evidence

Contract	Summary
Outcome	Apply privacy, exact money, persistence, atomicity, synthetic-data safety, and evidence rules across all features.
UI/API/data	Backend authorization and PostgreSQL are authoritative; monetary values use exact decimal strings; rejected writes leave no partial state.
Dependencies/gates	Continuous across FEAT-01–FEAT-14; unresolved decisions remain explicit gates.
DoD/evidence	STS-INT/UI and the 43-row AC-to-test map; preserve TDD evidence and tested commits for meaningful changes.

Feature-to-requirement traceability

Feature	Supporting SRS baseline	Acceptance criteria
FEAT-01	FR-01, FR-02; BR-11; NFR-07	F1-AC-01–06
FEAT-02	FR-02, FR-03; BR-08, BR-11; NFR-01, NFR-07	F2-AC-01–06
FEAT-03	FR-14; BR-01; NFR-02, NFR-05	F3-AC-01–05
FEAT-04	FR-05; BR-05, BR-06, BR-10; NFR-02, NFR-04, NFR-05	F4-AC-01–05
FEAT-05	FR-07–FR-09; BR-03, BR-04, BR-06, BR-07, BR-10; NFR-02–NFR-04	F5-AC-01–05
FEAT-06	FR-06, FR-08, FR-09; BR-02, BR-04, BR-06, BR-07, BR-10; NFR-02–NFR-04	F6-AC-01–05
FEAT-07	FR-09, FR-10; BR-05–BR-07; NFR-02, NFR-06	F7-AC-01–05
FEAT-08	FR-02; BR-04, BR-08, BR-10, BR-11; NFR-01–NFR-07	F8-AC-01–06

Acceptance criteria

Acceptance ID	Acceptance criterion
F1-AC-01	Correct synthetic credentials for each seeded role establish a session that the server accepts on a subsequent request.
F1-AC-02	Invalid credentials establish no authenticated session and return a user-safe failure.
F1-AC-03	Current identity returns only the identity associated with the validated session; changing client-supplied identity data cannot impersonate another User.
F1-AC-04	Logout makes the session unusable for a subsequent protected request.
F1-AC-05	Session lifecycle, cookie/trusted-origin protections, and credential hashing are documented and verified without committing credentials or exposing secret values.
F1-AC-06	A disabled User cannot establish a new session; disabling an active User revokes stored sessions immediately, and the same session cannot access identity/protected operations afterward.
F2-AC-01	A User can access their own authorized Sprint 1 financial operations, while every financial operation derives ownership from the session.
F2-AC-02	Attempts to read or mutate another User's Account, Transaction, allocation, balance, summary, or history are denied without exposing record data or changing either User's state.
F2-AC-03	An Administrator can perform the approved platform User enable/disable operations and receives no financial fields or aggregates in those responses.
F2-AC-04	User and unauthenticated actors cannot invoke Administrator access-management operations.

Continued on next page

Acceptance ID	Acceptance criterion
F2-AC-05	Administrator requests to financial APIs, including direct record lookup, list/filter, balance, category, summary, and history surfaces, are denied with no financial response.
F2-AC-06	Disabling a User immediately removes all stored sessions, blocks new session creation, and causes subsequent protected API requests using pre-existing cookies to be rejected.
F3-AC-01	A fresh prepared demo database contains the synthetic roles and required Cash, Loan Taken, Income, and at least two Expense records needed for the Sprint 1 journey.
F3-AC-02	Seeded financial records have explicit owners and are isolated between synthetic Users.
F3-AC-03	Re-running the seed does not duplicate demo fixtures or reset unrelated/user-created data.
F3-AC-04	Seed/demo/test data contains no real personal/financial data, plaintext stored passwords, committed reusable credentials, or secrets.
F3-AC-05	The seeded Administrator can manage access but cannot read or receive financial data.
F4-AC-01	The owner can record an exact opening balance on their seeded Cash Account, and a subsequent read returns the persisted value.
F4-AC-02	The opening balance establishes the Cash starting position without creating Income/Expense activity.
F4-AC-03	A foreign-owned, Administrator-owned, or non-Cash Account cannot be used for this User opening-balance action; rejected attempts leave state unchanged.
F4-AC-04	A Sprint 1 Transaction requires an explicit `YYYY-MM-DD` date that round-trips unchanged; no timezone conversion or implicit current-date default occurs.
F4-AC-05	THB values use exact minor units, serialize with two fractional digits, and reject more than two fractional digits without rounding; opening amounts must be positive.
F5-AC-01	A valid salary receipt increases the selected Cash balance by the exact Transaction total and increases the selected Income category total by the exact allocation amount.
F5-AC-02	Salary is persisted as a dated Transaction with its allocation and remains visible after a database-backed reload/restart check.
F5-AC-03	The allocation total must exactly equal the Transaction total; a mismatch or no allocation is rejected with no partial Transaction, allocation, balance, or summary effect.
F5-AC-04	A wrong-type, foreign-owned, or otherwise unauthorized receiving/allocation Account is rejected without exposing or changing another User's data.
F5-AC-05	The salary receipt is classified as Income only; its Cash effect is not counted again as another Income or Expense.
F6-AC-01	A valid Sprint 1 credit-card purchase with one or more Expense allocation lines increases card debt by the exact total and each Expense category by its line amount. The demonstration also proves a purchase split across at least two Expense Accounts.
F6-AC-02	A valid repayment decreases the selected Cash balance and Loan Taken debt by the exact repayment amount and creates no Income/Expense allocation effect.
F6-AC-03	The purchase and repayment Transactions and all allocation lines persist and appear in owner-scoped history.
F6-AC-04	Purchase/repayment allocation mismatch, missing lines, invalid target type, or foreign-owned reference is rejected with no partial record or balance/category effect.
F6-AC-05	The full salary → split purchase → repayment journey preserves exact arithmetic and does not classify borrowing/repayment as Income or Expense.
F7-AC-01	A User sees bank balance and card debt that match the persisted opening position and committed payment/receipt/charge/repayment effects.
F7-AC-02	Income/Expense summaries and per-category spending match their eligible persisted allocation lines; opening balances, transfers, borrowing, and loan repayment are excluded from income/expense totals.
F7-AC-03	Transaction history shows the User's persisted Sprint 1 records and allocations with their accepted dates, and excludes another User's records.
F7-AC-04	Results remain correct after a new server/database-backed read and the required restart/ persistence scenario; browser-only or mock-only values do not satisfy acceptance.
F7-AC-05	Sprint 1 uses all-time summaries, explicit `YYYY-MM-DD` dates, and exact THB values rendered with two fractional digits; no implicit rounding or date filter is applied.
F8-AC-01	No binary floating-point arithmetic is used for monetary transport, allocation equality, balance effects, persistence, or summaries.
F8-AC-02	Every financial mutation is atomic across Transaction, allocation lines, and derived balance/ category effects; rejected operations leave persistent state unchanged.
F8-AC-03	Authentication, role, and ownership tests execute at backend/API or HTTP boundaries; UI-only hiding/mocks are insufficient evidence.
F8-AC-04	Synthetic data and secret handling satisfy NFR-05/NFR-07; the reviewed diff and tracked files contain no real financial records, credentials, or secrets.
F8-AC-05	Every feature AC has one or more linked STS IDs; actual results and tested commits are recorded; TDD failing-test evidence exists for meaningful business-rule work.

Continued on next page

Acceptance ID	Acceptance criterion
F8-AC-06	The teammate review/approval of this contract occurs before Issue #8 feature implementation begins; both students participate in requirements/design, testing, AI-agent supervision, implementation, review, integration, deployment, and maintenance, with bounded task splits rather than permanent documentation/coding roles; project board, checklist, and PR target/status remain accurate.

Test strategy

Tests assert observable API/HTTP/UI results and database state, not private implementation. Use PostgreSQL-backed API tests for ownership, business rules, atomicity, and persistence; HTTP/Auth tests for session behavior; UI tests for accessible visible behavior. Mock-only tests do not prove persistence or authorization.

Level	Seam and use	Evidence boundary
HTTP/Auth integration	Exercise Better Auth through the Elysia app's request handler and a disposable PostgreSQL database. Use real synthetic credentials and returned session cookies for login, identity, logout, and session/role lifecycle checks.	HTTP responses, cookie/session lifecycle, and subsequent protected-request outcomes. No listening dev server is needed.
API integration	Call the tRPC router with controlled session contexts and a real disposable PostgreSQL-backed Prisma client for application operations.	Externally visible result/error and persisted owner-scoped state. Use a fake/minimal database only for behavior that is explicitly independent of persistence.
Database integration	Use the real Prisma/PostgreSQL connection and versioned migrations for persistence, exact values, allocation atomicity, and restart/re-read behavior.	Committed records and aggregates read back from PostgreSQL. Never run destructive reset commands against an unknown database.
UI component/ workflow	Render user-visible React components/routes with the existing Bun and Testing Library setup. Use this for field validation, accessible feedback, and display of API-backed result states.	Accessible controls and visible content/ actions. A mocked UI result does not prove backend permission or persistence.
Repository/ security review	Inspect tracked seed/config/test artifacts and the tested commit.	No real financial data, reusable credentials, plaintext stored passwords, or secrets. This supplements automated tests.

Prior art: tRPC caller tests in `packages/api/src/routers/index.test.ts`, Elysia `app.handle()` tests in `apps/server/src/app.test.ts`, PostgreSQL tests in `packages/db/src/index.test.ts`, and React Testing Library in `apps/web/src/components/button.test.tsx`.

Planned software test cases

The STS defines 56 cases across authentication, authorization, seeding, opening balances/dates, salary, card purchase/ repayment, financial views, integrity, and UI behavior. Each row records setup/action, expected result, test seam, and covered acceptance IDs.

Test ID	Level	Setup and action	Expected result	Acceptance IDs
STS-AUTH-01	HTTP/ Auth + DB	Active synthetic User A exists with a correctly provisioned password. Submit valid credentials, then use the returned session for current identity and a protected request.	Authentication succeeds; a subsequent request resolves User A; no caller-supplied identity is needed.	F1-AC-01, F1-AC-03
STS-AUTH-02	HTTP/ Auth + DB	Active synthetic Administrator exists. Submit valid credentials and request identity.	Authentication succeeds and identity reflects the Administrator, without granting finance access (finance denial is also asserted in STS-AUTHZ-08).	F1-AC-01
STS-AUTH-03	HTTP/ Auth	Use an unknown email and a known email with an incorrect password.	Both fail without an authenticated session; response/ log output contains no submitted password or secret.	F1-AC-02
STS-AUTH-04	HTTP/ Auth + API	Authenticate User A, log out using that session, then repeat a protected request with the same cookie/token.	Logout succeeds; the subsequent request is unauthenticated/ rejected.	F1-AC-04

Continued on next page

Test ID	Level	Setup and action	Expected result	Acceptance IDs
STS-AUTH-05	API	Authenticate as User A; attempt to supply User B's identity/owner ID in a current-identity or protected request.	Returned identity and authorization remain tied to User A's server-validated session; User B is not impersonated.	F1-AC-03
STS-AUTH-06	DB + repository review	Provision a test password and inspect only its stored credential representation, then authenticate with the original synthetic password.	The database does not contain the plaintext password; authentication still verifies correctly. Do not print or persist the credential in test output.	F1-AC-05, F8-AC-04
STS-AUTH-07	HTTP/ Auth + DB	Mark a synthetic User disabled before a fresh login; submit that User's valid credentials.	Sign-in fails without issuing a session cookie; no protected access is granted.	F1-AC-06, F2-AC-06
STS-AUTH-08	HTTP/ Auth + API + DB	Establish a session, disable that User through the Administrator operation, then request identity and a protected operation using the pre-existing cookie.	The stored session is deleted atomically with disablement; identity resolves to no session and protected operations are rejected immediately. The stale cookie alone grants no access.	F1-AC-06, F2-AC-06
STS-AUTH-09	HTTP/ Auth + configuration review	Inspect actual configured session lifecycle and cookies; exercise an untrusted Origin and an allowed Origin; check cookie attributes and configured expiry/renewal behavior.	Documentation matches the running configuration; only trusted origins are accepted; cookies are not available to client-side script and retain required deployed security attributes. Expiry/renewal assertions use the selected config, not an invented duration.	F1-AC-05
STS-AUTH-10	HTTP/ Auth + DB	Sign in as a seeded User while submitting a client-controlled `role: Administrator` field; then request the identity using the returned session.	The login either ignores the unsupported field or rejects it; in no case does the User session gain Administrator role. The persisted server-assigned role remains authoritative.	F1-AC-03, F8-AC-03
STS-AUTH-11	HTTP/ Auth + DB concurrency	Deterministically pause sign-in after its enabled-status read; let an Administrator disable the User and commit, then resume session creation. After sign-in completes, re-enable the User and use the returned cookie.	No session row created by the racing sign-in remains; the returned cookie resolves to no identity after re-enable and cannot revive the revoked session.	F1-AC-06, F2-AC-06, F8-AC-03
STS-AUTHZ-01	API	Call each implemented protected financial/admin operation with no session.	Each protected operation is rejected as unauthenticated; no records or protected values are returned.	F2-AC-04, F8-AC-03
STS-AUTHZ-02	API + DB	Authenticate User A and read/ mutate an owned Sprint 1 record using its legitimate ID.	The operation succeeds within the User A scope and changes only User A's intended state.	F2-AC-01
STS-AUTHZ-03	API + DB	Authenticate User A; query User B's Account/Transaction by direct ID and through any available list/filter/summary inputs.	Requests are denied or return the documented non-disclosing empty/not-found result; no User B data is present in responses.	F2-AC-02
STS-AUTHZ-04	API + DB	Authenticate User A; attempt to update/delete User B's record by ID. Compare both Users' persistent state before and after.	The attempt is denied; neither User's record, balance, allocation, nor summary changes.	F2-AC-02
STS-AUTHZ-05	API + DB	Authenticate User A; submit a valid-looking Transaction that references User B's source or allocation Account.	Request is denied without disclosure; no Transaction, allocations, or financial effects are committed.	F2-AC-02, F8-AC-02, F8-AC-03

Continued on next page

Test ID	Level	Setup and action	Expected result	Acceptance IDs
STS-AUTHZ-06	API	Call Administrator access-management operations as User A and with no session.	Both are denied; no platform User access state changes.	F2-AC-04, F8-AC-03
STS-AUTHZ-07	API + DB	Authenticate as Administrator; list synthetic Users, enable/ disable one, then inspect the response and persistent access state.	Approved platform access operations succeed; only required identity/access fields are returned; no financial entity or aggregate is joined, serialized, or exposed.	F2-AC-03, F3-AC-05
STS-AUTHZ-08	API + DB	Authenticate as Administrator; attempt each available finance route/category: Account list/ detail, Transaction list/detail, create/update, balance, category totals, summary, and history.	Every financial operation is denied and returns no financial information; no mutation occurs. Cover direct IDs and generic lookup surfaces, not just hidden UI links.	F2-AC-05, F3-AC-05, F8-AC-03
STS-AUTHZ-09	API + DB	Disable a User who has an active stored session, then invoke a protected API operation with that session context/cookie.	The disable operation deletes persisted sessions and the protected operation rejects access; no financial data is returned or changed.	F2-AC-06, F8-AC-03
STS-SEED-01	DB + API	Prepare a fresh disposable database and run the documented seed operation.	Required synthetic User and Administrator identities, bank/ Cash Account, Loan Taken/card Account, Income Account, and at least two Expense Accounts are available for the journey.	F3-AC-01
STS-SEED-02	DB + API	Query seeded finance records as User A and User B using their respective sessions.	Each User sees only their own seeded Financial Accounts/ categories; owner identity cannot be changed by request input.	F3-AC-02
STS-SEED-03	DB	Run the seed twice, with a separate synthetic user-created record present between runs.	Demo identities/Accounts/ categories are not duplicated; the unrelated record and its state are not reset or deleted.	F3-AC-03
STS-SEED-04	Repository/ security review	Inspect seed fixtures, examples, environment files, logs captured by tests, and the changed-file diff.	Data is synthetic; no real personal/financial data, reusable committed credentials, plaintext stored password, or secrets are present.	F3-AC-04, F8-AC-04
STS-OPEN-01	API + DB	User A records an allowed exact opening amount on their seeded bank/Cash Account, then reads the Account back.	The exact opening position is persisted and returned; it is the starting Cash amount for subsequent balance calculations.	F4-AC-01
STS-OPEN-02	API + DB	Record an opening Cash position with no Transactions and query Income/Expense totals.	Opening balance affects the Cash position only; it creates no Income or Expense allocation/ summary activity.	F4-AC-02
STS-OPEN-03	API + DB	Try to set an opening balance on a foreign-owned Account, a non-Cash Account, or as Administrator. Also repeat after financial postings if the operation would rewrite a baseline.	Each unauthorized/wrong-type/ out-of-scope change is rejected and persistent state remains unchanged. Apply the approved one-time/baseline policy; do not infer correction behavior.	F4-AC-03, F8-AC-02, F8-AC-03
STS-OPEN-04	API + DB	Create a Sprint 1 Transaction with explicit `YYYY-MM-DD`, then read it back in history.	The date round-trips unchanged; invalid/missing dates are rejected, and no default or timezone shift is applied.	F4-AC-04, F7-AC-03, F7-AC-05
STS-OPEN-05	API + DB	Submit `1.00` THB and `0.01` THB, then submit `1.001` THB.	Valid amounts remain exact; `1.001` is rejected; values are not silently rounded, and serialization uses two fractional digits.	F4-AC-05, F7-AC-05, F8-AC-01

Continued on next page

Test ID	Level	Setup and action	Expected result	Acceptance IDs
STS-OPEN-06	API + DB	Exercise zero and negative opening amounts.	Both are rejected with persistent Account state unchanged. No arbitrary maximum or allocation-line limit is asserted.	F4-AC-05
STS-RECEIVE-01	API + DB	User A submits a valid receive-money Transaction to their Cash Account with one Salary/Income allocation exactly equal to the total and an approved explicit date.	Cash increases by the total; the Income category increases by its allocation; the Transaction/ allocation persist and are returned by a later read.	F5-AC-01, F5-AC-02, F5-AC-05, F8-AC-01
STS-RECEIVE-02	API + DB	Submit a receive-money Transaction whose allocation sum differs from its total. Read all affected records/ balances before and after.	Request is rejected; no Transaction/allocation or balance/category/summary effect remains.	F5-AC-03, F8-AC-02
STS-RECEIVE-03	API + DB	Submit a receive-money Transaction with no allocation lines.	Request is rejected and persistent financial state is unchanged.	F5-AC-03, F8-AC-02
STS-RECEIVE-04	API + DB	Use a non-Cash receiver, non-Income destination in the Sprint 1 salary flow, or another User's Account.	Request is rejected without exposing or changing foreign records; no partial financial effect occurs.	F5-AC-04, F8-AC-02, F8-AC-03
STS-RECEIVE-05	API + DB	Submit valid THB strings with zero, one, and two fractional digits, plus an amount with three fractional digits.	Valid inputs are interpreted exactly and returned with two fractional digits; excess precision is rejected without rounding or a persistent effect.	F5-AC-01, F5-AC-03, F8-AC-01
STS-RECEIVE-06	DB + API	Complete salary receipt, close/ recreate the API/database client (and run the required application/database restart scenario), then query the record and balances again.	The Transaction, allocation, Cash balance effect, and Income total remain available and consistent from PostgreSQL.	F5-AC-02, F7-AC-04, F8-AC-02
STS-CARD-01	API + DB	User A selects their Loan Taken/card Account and posts a valid purchase total `C` split across distinct Expense allocations `E1` and `E2`, where `E1 + E2 = C`.	Card debt increases by `C`; each Expense category increases by its line amount; the dated Transaction and both allocations persist.	F6-AC-01, F6-AC-03, F6-AC-05, F8-AC-01
STS-CARD-02	API + DB	In the Sprint 1 card-purchase flow, submit a mismatched allocation total, no allocations, a non-Expense allocation, or a foreign-owned allocation.	Invalid case is rejected; no purchase, allocation, debt, category, summary, or history change is committed.	F6-AC-04, F8-AC-02, F8-AC-03
STS-CARD-03	API + DB	With a valid known card debt and Cash position, submit a Cash-funded repayment `R` allocated to the owned Loan Taken/card Account.	Cash decreases by `R`; card debt decreases by `R`; no Income or Expense total changes because of repayment; the repayment persists.	F6-AC-02, F6-AC-03, F6-AC-05, F8-AC-01
STS-CARD-04	API + DB	Submit repayment using a wrong source/destination Account type or another User's Account.	Request is rejected with no partial record or financial effect and no foreign data disclosure.	F6-AC-04, F8-AC-02, F8-AC-03
STS-CARD-05	DB + API	Run the complete isolated sequence: opening balance `O`, salary `S`, split card purchase `C`, repayment `R`; read all balances and categories through the API after writes.	`Cash = O + S - R`; card debt equals starting debt plus `C - R`; Income increases only by salary; Expense categories increase by `E1`/`E2`; each result matches persisted records. Values use an approved valid fixture.	F6-AC-03, F6-AC-05, F7-AC-01, F7-AC-02, F8-AC-01

Continued on next page

Test ID	Level	Setup and action	Expected result	Acceptance IDs
STS-CARD-06	API + DB	Attempt a repayment exceeding current card debt and a Cash outflow exceeding the available balance; also submit zero/negative totals and allocations.	Each attempt is rejected and leaves Transaction, allocation, debt, Cash balance, category totals, and history unchanged.	F6-AC-04, F8-AC-02
STS-CARD-07	API + DB	User A posts a valid Sprint 1 card purchase with exactly one Expense allocation equal to the Transaction total, using a valid fixture amount. Read persisted debt and category totals before and after.	Purchase succeeds and persists; card debt increases by the total and the selected Expense category increases by that same line amount. No second allocation is required for this valid one-line purchase.	F6-AC-01, F6-AC-03, F8-AC-01
STS-VIEW-01	API + DB	Complete known opening/ receipt/purchase/repayment fixture, then request User A's balances.	Cash and Loan Taken balances match the opening position and each committed posting exactly; values are sourced from persisted state.	F7-AC-01, F8-AC-01
STS-VIEW-02	API + DB	Query User A's all-time Income/Expense summary after the opening position, salary, split purchase, and repayment.	Income includes salary allocation; expenses include only Expense allocations; opening position and loan repayment are excluded. Exact totals match persisted allocation lines.	F7-AC-02, F7-AC-05, F6-AC-05
STS-VIEW-03	API + DB	Query category spending and transaction history as User A after the full journey.	Dining/grocery category totals match the split allocations; salary, purchase, and repayment history belongs to User A, includes allocations, and uses approved dates.	F7-AC-02, F7-AC-03
STS-VIEW-04	API + DB	Authenticate User A and query User B's balance, category, summary, or history via IDs/ filters.	The response exposes no User B financial values or record data; no mutation occurs.	F7-AC-03, F8-AC-03
STS-VIEW-05	DB + API	Complete the journey, then perform a fresh database-backed read after the application/client is restarted.	Balances, summaries, categories, and history still match committed PostgreSQL records.	F7-AC-04, F5-AC-02, F6-AC-03, F8-AC-02
STS-VIEW-06	API + DB	Query the Sprint 1 summary and category totals without a date filter; verify returned amounts include exactly two fractional digits.	The result includes all persisted eligible allocations with no implicit period/date filter and no rounding; any date-filtered reporting remains outside Sprint 1.	F7-AC-05
STS-INT-01	API + DB	Submit a financially invalid multi-allocation write and compare the complete persistent state before/after. Include a failure case affecting an otherwise valid source/ target pair.	The operation is rejected and there is no partial Transaction, allocation, balance, category, summary, or history effect.	F5-AC-03, F6-AC-04, F8-AC-02
STS-INT-02	API + DB	Run exact THB arithmetic/ equality/balance cases including repeated `0.01` allocations, a mismatch, and serialization round-trip.	Exact minor-unit totals remain exact through API, persistence, and all-time summary; values render to two fractional digits and no binary-float drift or rounding occurs.	F4-AC-05, F5-AC-01, F6-AC-01, F8-AC-01
STS-INT-03	API/ HTTP + repository review	Cause invalid login, authorization denial, and financial validation failure; inspect client responses, tracked fixtures, and captured logs.	No password, secret, or financial record belonging to another User is exposed in error output/logs; tracked sample data remains synthetic.	F1-AC-02, F2-AC-02, F3-AC-04, F8-AC-04

Continued on next page

Test ID	Level	Setup and action	Expected result	Acceptance IDs
STS-UI-01	UI component/ workflow	Render the login UI, submit empty/invalid fields and then valid synthetic credentials through the intended auth interaction.	Inputs have accessible labels; field/server errors are visible; successful authentication navigates to the authenticated experience without exposing credentials. This does not replace STS-AUTH cases.	F1-AC-01, F1-AC-02
STS-UI-02	UI workflow + API-backed test	Render the authenticated User dashboard with persisted API results from the complete fixture.	Visible bank balance, card debt, summary, category spending, and history match the API-backed values. Test loading/error/empty states without treating mocked values as persistence evidence.	F7-AC-01, F7-AC-02, F7-AC-03, F7-AC-04, F7-AC-05
STS-UI-03	UI component/ workflow + API	Render the Administrator experience and inspect available actions/navigation; attempt the finance API separately as STS-AUTHZ-08.	Administrator access controls are available; financial views/ navigation are not presented as allowed capabilities. The server-side denial remains authoritative regardless of UI.	F2-AC-03, F2-AC-05, F8-AC-03
STS-UI-04	UI finance-cache boundary	Cache a financial query response under User A's authenticated identity, then request the same finance query for User B using the same QueryClient.	User B cannot read User A's cached financial response; the query still targets the same tRPC procedure and input. A browser-level account-switch journey remains separate evidence.	F7-AC-03, F8-AC-03

6 Part F: working Sprint 1 and verification

Release and demo records as of 8 November 2026; full-suite verification rerun on 9 November 2026.

6.1 Release record

The report assesses `main` at tag `sprint1-v1.0.0`, commit `e99da77c8181a97a32abfddc37b116ff297acf27`. Reviewed release PR #27 merged `staging-1` to `main`. Parts A through E summarize the required specifications; Part F links test, review, and release evidence. The STS and Issue #8 record tested commits and results.

6.2 Local setup

Follow the README setup instructions. Use Bun 1.4.4, Git, and Docker Compose.

1. Install Bun 1.4.4 and run `bun install`. Copy the server, web, and database environment schemas into ignored local files. Set `BETTER_AUTH_SECRET` to a random value of at least 32 characters and align the server and database `DATABASE_URL` values.
2. Start PostgreSQL with `bun run db:dev:up`. Generate Prisma Client and apply versioned migrations with `bun run db:generate` and `bun run db:migrate`. Set `POCKIT_DEMO_PASSWORD` only in ignored `apps/server/.env`, then run `bun run db:seed` to create synthetic users and hashed passwords. See the database Compose file.
3. Sign in with one of the seeded accounts: `demo.user@pockit.test`, `demo.user2@pockit.test`, or `demo.admin@pockit.test`. Use the password set in the ignored environment file. Start the apps yourself with `bun run dev`. The web app listens on http://localhost:3001; the API listens on http://localhost:3000. The assistant did not start either app.
4. Point `DATABASE_URL` at a disposable migrated test database, then run `bun run test`. The test suite requires PostgreSQL. Run `bun run check-types`, `bun run build`, and `bun run check` for the other workspace checks.

6.3 Synthetic finance demonstration

Use the seeded primary User and synthetic accounts. Start from the seeded account state, then follow the five steps. The matching contracts appear in Part E; the seed test is `apps/server/src/seed-data.test.ts`.

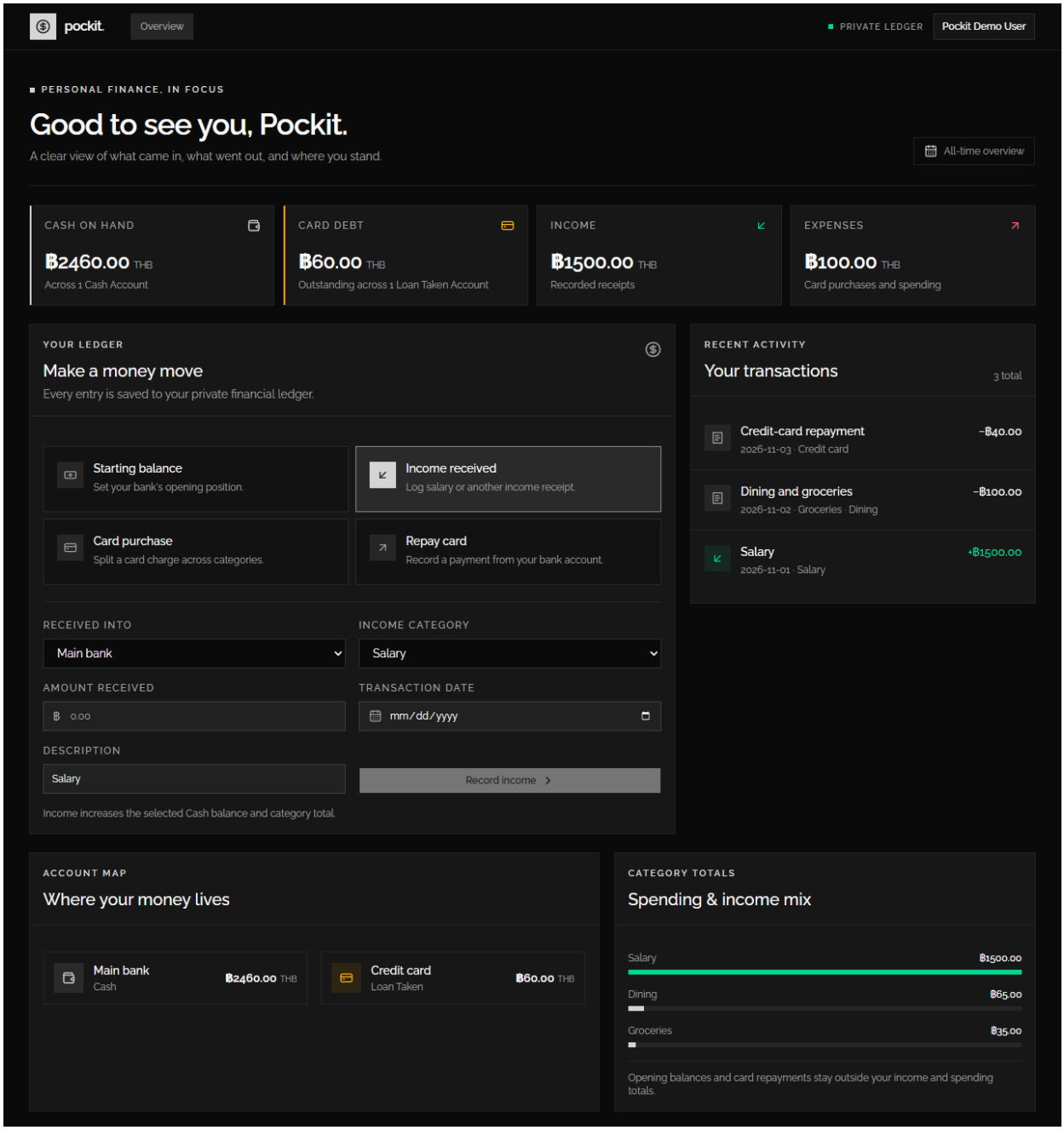
Step	Action	Expected persisted outcome
1	Set the Main bank Cash opening balance to THB 1,000.00.	Cash balance is THB 1,000.00. The opening position is not an Income or Expense Transaction.
2	Receive THB 1,500.00 salary into Main bank, dated 2026-11-01, allocated to Salary Income.	Cash is THB 2,500.00; Income total is THB 1,500.00.
3	Record a THB 100.00 Credit card purchase dated 2026-11-02, split into Dining THB 65.00 and Groceries THB 35.00.	Card debt is THB 100.00; Expense total is THB 100.00 with the two category totals preserved.
4	Repay THB 40.00 from Main bank to Credit card, dated 2026-11-03.	Cash is THB 2,460.00; card debt is THB 60.00. The repayment is not new Income/Expense.
5	Reload the financial views and retrieve history.	Persisted history contains the three dated Transactions and allocation lines; the opening position remains a baseline, not a Transaction. Expected final totals: Cash 2,460.00; card debt 60.00; Income 1,500.00; Expense 100.00; Dining 65.00; Groceries 35.00.

6.4 Required student screenshots and screen dumps

The assignment requires screenshots of the normal finance journey, a rejected operation, and authentication or forbidden access. The captures below use synthetic records and show no credentials. They supplement the automated tests. See the assignment specification.

F-1: normal finance journey and persisted result

Acceptance criteria: FEAT-04 to FEAT-07; F4-AC-01, F5-AC-01/02, F6-AC-01/02/03, F7-AC-01 to F7-AC-04

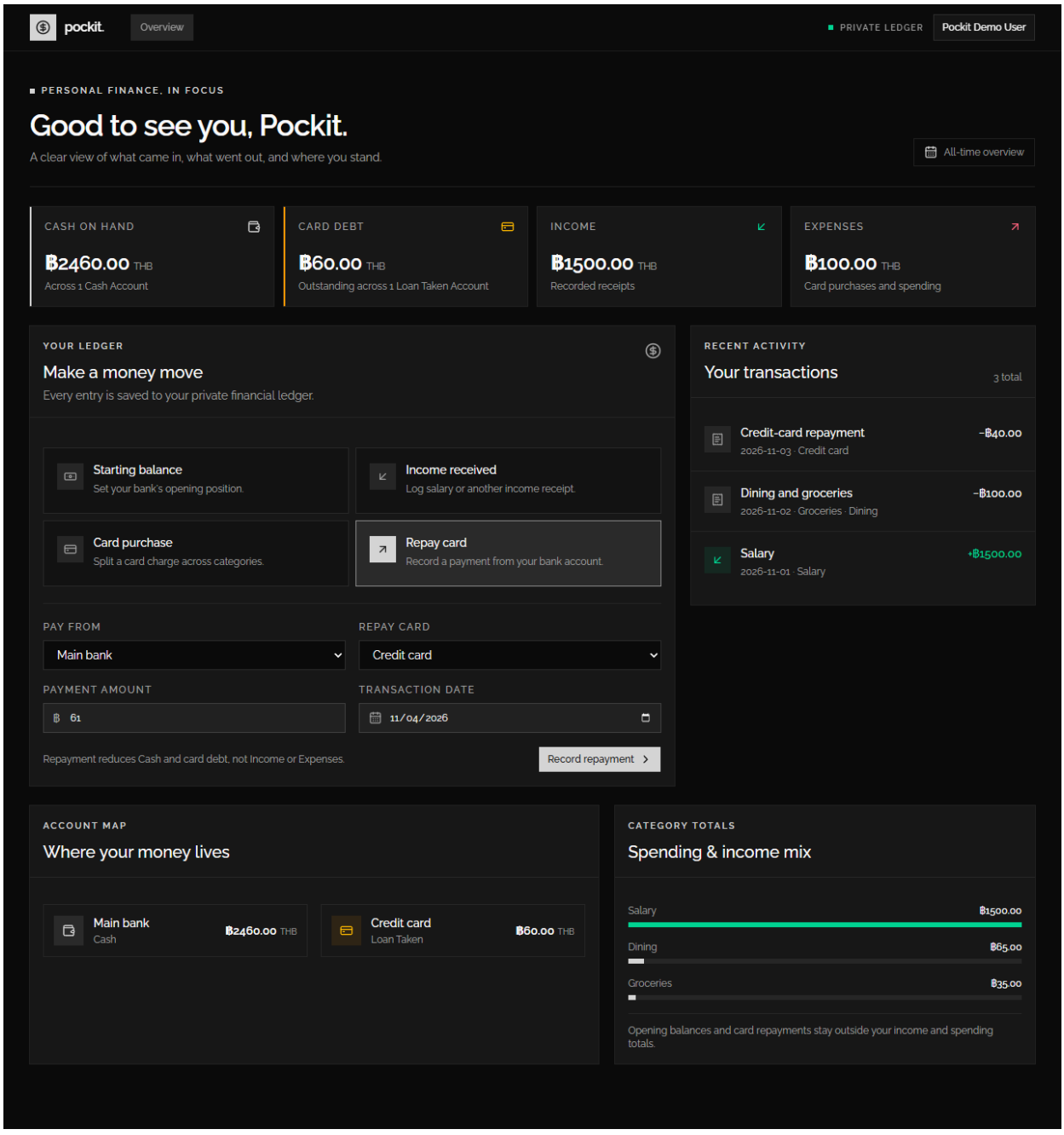


The signed-in synthetic User's dashboard was reloaded before capture. It shows Cash THB 2,460.00, card debt THB 60.00, Income THB 1,500.00, Expenses THB 100.00, and Dining/Groceries totals of THB 65.00/THB 35.00. The three dated Transactions remain in history.

Automated tests: The API test at `packages/api/src/routers/finance.test.ts` and fresh-client test at `apps/server/src/finance-persistence.test.ts` verify the persisted journey and balances.

F-2: rejected financial operation

Acceptance criteria: FEAT-05/06; F5-AC-03, F6-AC-04, F8-AC-02



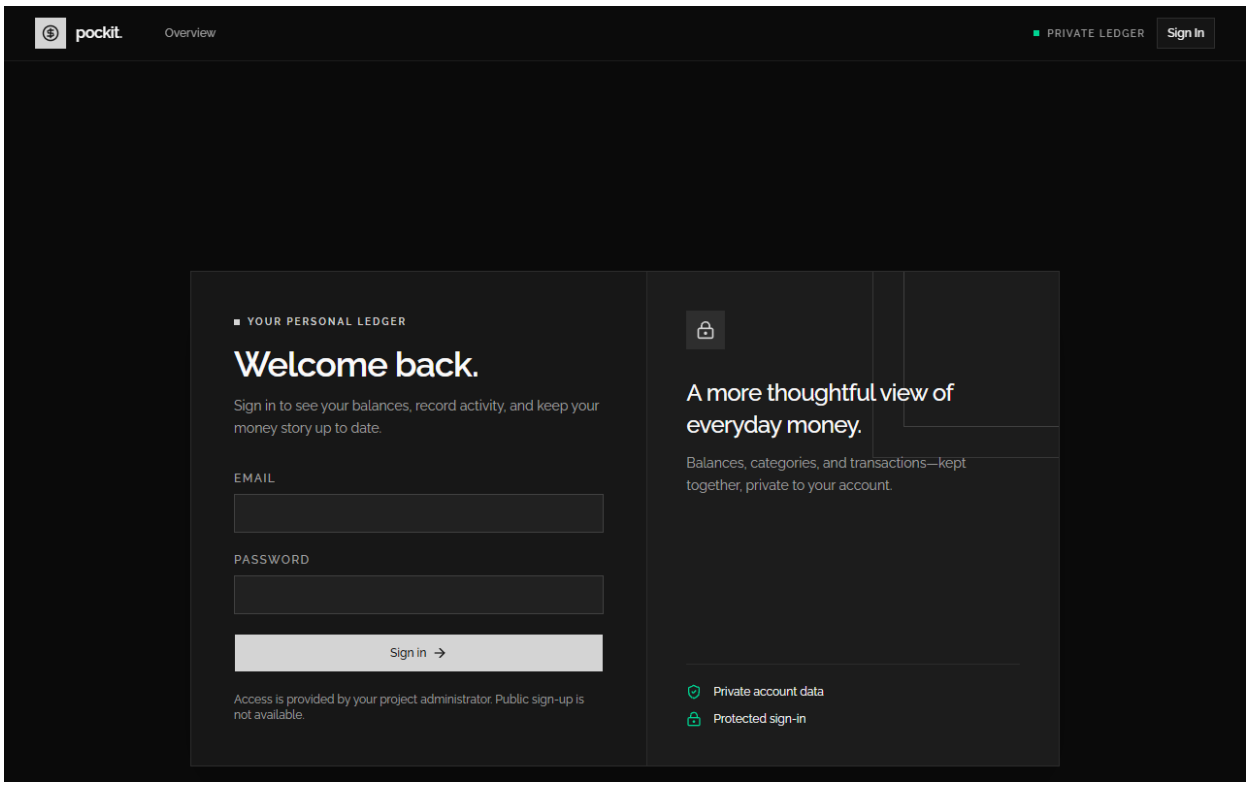
The synthetic User submitted a THB 61.00 card repayment when outstanding debt was THB 60.00. The API returned HTTP 400 with `Loan repayment cannot exceed outstanding debt`. The screenshot shows the unchanged Cash/card balances and three-Transaction history after rejection.

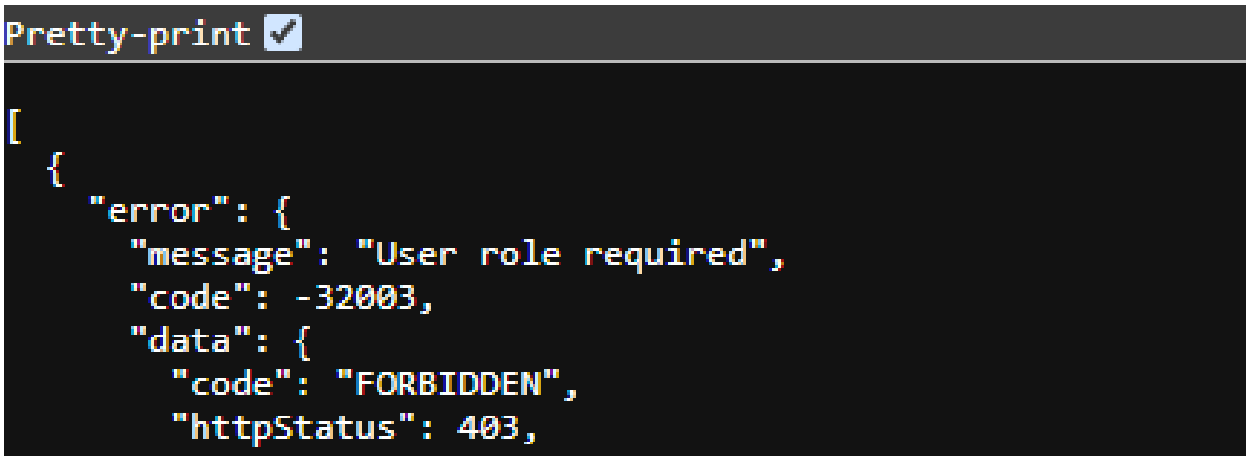
Automated tests: The API test at `packages/api/src/routers/finance.test.ts` rejects invalid transactions and confirms that transaction counts, allocations, summaries, and balances remain unchanged.

F-3: sign-out and Administrator finance denial

Acceptance criteria: FEAT-01/02; F1-AC-01 to F1-AC-04, F1-AC-06, F2-AC-02/05

The first screenshot shows the login form after the synthetic User signed out. The second shows the Administrator's live request for `finance.summary.get`, which returned HTTP 403, `FORBIDDEN`, and `User role required`. The response image is cropped to those fields; it contains no credentials or finance records.





Automated tests at `apps/server/src/auth.test.ts` cover valid/invalid credentials, current identity, and logout. Finance access tests at `packages/api/src/routers/finance.test.ts` reject cross-User and Administrator access.

6.5 Recorded build, test, and migration results

The full suite was rerun on 9 November 2026 with Bun 1.4.4 against an isolated PostgreSQL 17 database. The tested commit is `d5abf73deae8a108364d51cf766a65c26b0759b6`; its Git tree matches the released tag `sprint1-v1.0.0`. The temporary database used port 55441 and was removed after the run. The screenshots came from a separate student-run browser walkthrough; the assistant did not start or stop a development server.

Check	Command / conditions recorded	Result
Full workspace tests	`DATABASE_URL=<isolated> mise exec -- bun run test`; Bun 1.4.4, PostgreSQL 17, four migrations.	PASS: 51 total; 51 pass, 0 fail, no skipped tests reported. API 21, server 14, web 14, DB 2.
Workspace type-check/build tasks	`VITE_SERVER_URL=http://localhost:3000 DATABASE_URL=<disposable> bun run check-types`.	PASS: all 6 Turbo tasks; web production client/SSR/Nitro build completed. Existing Rolldown module-directive warnings were non-fatal.
Direct API regression	`cd packages/api; bun test --timeout 5000`.	PASS: 21 tests in one Bun process on Bun 1.4.4; reported duration 2.80 seconds.
Session-race regression	Targeted `apps/server/src/auth.test.ts` against disposable PostgreSQL; full command in Part E.	PASS: 10 tests, 43 assertions. Without the guard, the test first found one remaining session instead of zero.
Versioned migrations	`DATABASE_URL=<isolated> mise exec -- bun run db:migrate:deploy`.	PASS: all four checked-in migrations applied to the disposable test DB.
Targeted style/diff checks	Targeted Biome on changed source/config files; `git diff --check`.	PASS on recorded targeted checks. A prior project-setup baseline `bun run check` failed with pre-existing diagnostics and is not reported as a pass.

Full command output is retained in `evidence/workspace-test-output.txt`. The excerpt shows the package summaries; Bun reported no skipped tests.

```
server:test:  14 pass
server:test:  0 fail
server:test: Ran 14 tests across 4 files. [7.03s]
@Pockit/api:test:  21 pass
@Pockit/api:test:  0 fail
@Pockit/api:test: Ran 21 tests across 4 files. [1398.00ms]
web:test:  14 pass
web:test:  0 fail
web:test: Ran 14 tests across 5 files. [37.32s]
@Pockit/db:test:  2 pass
@Pockit/db:test:  0 fail
@Pockit/db:test: Ran 2 tests across 1 file. [238.00ms]
```

6.6 Acceptance-criteria to actual-test crosswalk

The table maps all 43 acceptance criteria to planned STS IDs, actual test evidence, and results. The register links each evidence key to a test file or project record. Part E contains test conditions and limitations.

Actual test evidence register

Evidence keys resolve to source files and representative test cases or process artifacts.

Ref.	Actual test or process evidence
E01	`apps/server/src/auth.test.ts`: HTTP authentication and session tests
E02	`packages/api/src/routers/finance.test.ts`: finance authorization and ledger tests
E03	`packages/api/src/routers/admin.test.ts`: Administrator access-management tests
E04	`apps/server/src/seed-data.test.ts`: synthetic seed tests
E05	`packages/api/src/finance/value.test.ts`: exact-money and date validation tests
E06	`packages/api/src/routers/finance.test.ts`: database-backed finance journey and rejection tests
E07	`apps/server/src/finance-persistence.test.ts`: fresh-client persistence test

Continued on next page

Ref.	Actual test or process evidence
E08	apps/web/src/components/finance-dashboard.test.tsx: finance UI component tests
E09	apps/web/src/components/sign-in-form.test.tsx: login UI tests
E10	apps/web/src/components/admin-user-management.test.tsx: Administrator UI test
E11	apps/web/src/utils/user-scoped-query-key.test.ts: cross-User cache isolation test
E12	docs/testing/sprint1-software-test-specification.md: full AC/test mapping and execution record
E13	Issue #5; Issue #7; Issue #8; PR #20; PR #22; PR #27: review, integration, release, and board links

All 43 stable criteria are retained. Evidence keys resolve above; outcomes follow the recorded STS result.

AC	Planned STS IDs	Actual evidence ref.	Result
F1-AC-01	STS-AUTH-01, STS-AUTH-02, STS-UI-01	E01, E09	PASS
F1-AC-02	STS-AUTH-03, STS-UI-01, STS-INT-03	E01, E09, E02, E04	PARTIAL
F1-AC-03	STS-AUTH-01, STS-AUTH-05, STS-AUTH-10	E01	PASS
F1-AC-04	STS-AUTH-04	E01	PASS
F1-AC-05	STS-AUTH-06, STS-AUTH-09	E01	PASS
F1-AC-06	STS-AUTH-07, STS-AUTH-08, STS-AUTH-11	E01	PASS
F2-AC-01	STS-AUTHZ-02	E02	PASS
F2-AC-02	STS-AUTHZ-03, STS-AUTHZ-04, STS-AUTHZ-05, STS-INT-03	E02, E01, E04	PASS
F2-AC-03	STS-AUTHZ-07, STS-UI-03	E03, E10	PARTIAL
F2-AC-04	STS-AUTHZ-01, STS-AUTHZ-06	E02, E03	PASS
F2-AC-05	STS-AUTHZ-08, STS-UI-03	E02, E03, E10	PARTIAL
F2-AC-06	STS-AUTH-07, STS-AUTH-08, STS-AUTHZ-09, STS-AUTH-11	E01, E03	PASS
F3-AC-01	STS-SEED-01	E04	PASS
F3-AC-02	STS-SEED-02	E04	PASS
F3-AC-03	STS-SEED-03	E04	PASS
F3-AC-04	STS-SEED-04, STS-INT-03	E04, E01, E02	PASS
F3-AC-05	STS-AUTHZ-07, STS-AUTHZ-08	E03, E02	PASS
F4-AC-01	STS-OPEN-01	E06	PASS
F4-AC-02	STS-OPEN-02	E06	PASS
F4-AC-03	STS-OPEN-03	E02, E06	PASS
F4-AC-04	STS-OPEN-04	E05	PASS
F4-AC-05	STS-OPEN-05, STS-OPEN-06, STS-INT-02	E05, E06	PASS
F5-AC-01	STS-RECEIVE-01, STS-RECEIVE-05	E06, E05	PASS
F5-AC-02	STS-RECEIVE-01, STS-RECEIVE-06	E06, E07	PASS
F5-AC-03	STS-RECEIVE-02, STS-RECEIVE-03, STS-RECEIVE-05	E06, E05	PASS
F5-AC-04	STS-RECEIVE-04, STS-AUTHZ-05	E02, E06	PASS
F5-AC-05	STS-RECEIVE-01, STS-VIEW-02	E06, E08	PASS
F6-AC-01	STS-CARD-07, STS-CARD-01, STS-INT-02	E06, E08, E05	PASS
F6-AC-02	STS-CARD-03	E06	PASS
F6-AC-03	STS-CARD-01, STS-CARD-03, STS-CARD-05, STS-CARD-07	E06, E08, E07	PASS
F6-AC-04	STS-CARD-02, STS-CARD-04, STS-CARD-06	E06, E02	PASS
F6-AC-05	STS-CARD-01, STS-CARD-03, STS-CARD-05, STS-VIEW-02	E06, E08, E07	PASS
F7-AC-01	STS-CARD-05, STS-VIEW-01, STS-UI-02	E06, E07, E08	PASS
F7-AC-02	STS-CARD-05, STS-VIEW-02, STS-VIEW-03, STS-UI-02	E06, E07, E08	PASS
F7-AC-03	STS-OPEN-04, STS-VIEW-03, STS-VIEW-04, STS-UI-02, STS-UI-04	E05, E06, E08, E02, E11	PASS
F7-AC-04	STS-RECEIVE-06, STS-VIEW-05, STS-UI-02	E07, E08	PASS
F7-AC-05	STS-OPEN-04, STS-OPEN-05, STS-VIEW-02, STS-VIEW-06, STS-UI-02	E05, E06, E08	PASS
F8-AC-01	STS-OPEN-05, STS-RECEIVE-01, STS-CARD-01, STS-CARD-03, STS-CARD-05, STS-CARD-07, STS-VIEW-01, STS-INT-02	E05, E06, E08, E07	PASS
F8-AC-02	STS-AUTHZ-04, STS-AUTHZ-05, STS-OPEN-03, STS-RECEIVE-02, STS-RECEIVE-03, STS-CARD-02, STS-CARD-04, STS-INT-01	E02, E06	PASS

Continued on next page

AC	Planned STS IDs	Actual evidence ref.	Result
F8-AC-03	STS-AUTHZ-01, STS-AUTHZ-02, STS-AUTHZ-03, STS-AUTHZ-04, STS-AUTHZ-05, STS-AUTHZ-06, STS-AUTHZ-07, STS-AUTHZ-08, STS-AUTHZ-09, STS-AUTH-11, STS-RECEIVE-04, STS-CARD-02, STS-CARD-04, STS-VIEW-04, STS-UI-03, STS-UI-04	E02, E03, E01, E06, E10, E11	PASS
F8-AC-04	STS-AUTH-06, STS-SEED-04, STS-INT-03	E01, E04, E02	PASS
F8-AC-05	This AC→STS table plus actual test/result/commit record in Issue #8	E12	PARTIAL
F8-AC-06	Reviewed PR/issue history and project-board/checklist state before Issue #8 implementation	E13	PARTIAL

6.7 Selected excerpts of actual automated tests

Business-rule test. This test rejects an allocation mismatch, then checks that the database state did not change. The source is `packages/api/src/routers/finance.test.ts`, in the test named *rejects invalid transaction inputs without changing ledger state*. The test passed in the API and root suites.

```
for (const input of invalidWrites) {
  await expectRejectedWithCode(
    api.finance.transactions.create(input as never),
    "BAD_REQUEST",
  );
}

const [transactionsAfter, allocationsAfter, summaryAfter, bankAfter, cardAfter] =
  await Promise.all([
    db.financialTransaction.count({ where: { ownerId: ids.userA } }),
    db.allocationLine.count({ where: { ownerId: ids.userA } }),
    api.finance.summary.get(),
    api.finance.accounts.get({ accountId: accounts.bank }),
    api.finance.accounts.get({ accountId: accounts.card }),
  ]);
expect(transactionsAfter).toBe(transactionsBefore);
expect(allocationsAfter).toBe(allocationsBefore);
expect(summaryAfter).toEqual(summaryBefore);
expect(bankAfter.balance).toBe(bankBefore.balance);
expect(cardAfter.balance).toBe(cardBefore.balance);
```

Ownership test. This API test rejects a foreign User's Account lookup and an allocation to that User's Account. It confirms that neither User's transaction count changes. The source is `packages/api/src/routers/finance.test.ts`, in the test named *does not disclose another User account and rejects cross-owner writes.*

```
await expectRejectedWithCode(
  api.finance.accounts.get({ accountId: accounts.userBBank }),
  "NOT_FOUND",
);
await expectRejectedWithCode(
  api.finance.transactions.create({
    form: "RECEIVE_MONEY",
    sourceAccountId: accounts.bank,
    totalAmount: "1.00",
    transactionDate: "2026-11-04",
    allocations: [
      { kind: "INCOME", accountId: accounts.userBBank, amount: "1.00" },
    ],
  }),
  "BAD_REQUEST",
);
expect(await db.financialTransaction.count({ where: { ownerId: ids.userA } }))
  .toBe(countBeforeA);
expect(await db.financialTransaction.count({ where: { ownerId: ids.userB } }))
  .toBe(countBeforeB);
```

6.8 Selected issue and PR evidence

The short history is requirements/design, reviewed Sprint 1 contracts, implementation, then a reviewed release to `main`. The linked Issues and PRs hold the full discussion and change history.

Issue / PR	Branch and authorship	Substantive review / result
Issue #5 / PR #20	`docs/system-level-sds`; `staging-1` to `staging-1`. Author: Guntee.	Polwarit reviewed architecture, data model, privacy, and deployment. Requested OQ-02's excess-precision/no-rounding policy remain proposed until confirmation. Merged.

Issue #7 / PR #22	`docs/sprint1-engineering-contracts`; `staging-1` to `staging-1`. Author: Guntee.	Polwarit reviewed FEAT-01–FEAT-08 and the STS. Requested valid one-line purchases remain allowed while the demo still splits across two categories. Merged.
Issue #8 / PR #27	Sprint 1 release from `staging-1` to `main`.	Reviewed by Polwarit; released at `e99da77c8181a97a32abfddc37b116ff297acf27` and tagged `sprint1-v1.0.0`. Implementation and regression PRs are linked from Issue #8.

Board status: Issue #5 and Issue #7 are Done; Issue #8 remains Started while this report is finalized. See the Sprint 1 project board.

6.9 Contribution and AI-use record

Student	Repository-visible contribution
Guntee Doungmanee, 67070501003	Requirements, design, implementation, tests, and report authorship. See Issue #3–Issue #8 and PR #18–PR #27.
Polwarit Watthanahemmarat, 67070501067	Requirements/design and test review, implementation reviews, release review, and setup confirmation. See the reviews linked above.

AI-use record. Guntee used Pi's coding assistant, including bounded review agents, for requirements, implementation, tests, debugging, review, and report preparation. Prompts below are representative paraphrases, not a transcript. Guntee reviewed generated changes and test results; Polwarit provided independent reviews recorded in the linked PRs. The students made project decisions. AI output was not treated as test evidence.

Use case	Technique	Representative prompt (paraphrased)	Human check
Requirements	Source-grounded extraction and synthesis.	"Use Theme 7 in the assignment to draft the SRS, traceability, and sprint plan. Keep unconfirmed decisions open."	Compared with the assignment; reviewed through Issue #3–Issue #7 and PR #18–PR #22.
Code/tests	Code and test drafting, then targeted revision.	"Implement Issue #8's finance journey. Test persistence, invalid transactions, cross-User access, and Administrator denial."	Reviewed diffs and ran recorded tests; gaps remain listed in Part F and the STS.
Debug/review	Error-driven diagnosis and bounded specialist review.	"Find why the finance test stalls; verify the cause and whether the fix completes normally." "Review the PR for authorization and session-race defects; report actionable findings."	Reproduced and checked test behavior; human teammate reviews and dispositions are linked in PR #23–PR #27.
Report	Scripted extraction, ID checks, LaTeX generation, and PDF inspection.	"Condense Parts A–F into tables; preserve every stable ID, add links, and render the Mermaid diagrams."	Ran generator tests and LaTeX build; checked IDs, links, diagrams, screenshots, and page layout.

6.10 Limitations

- Browser-level route/role journeys and structured log-redaction assertions were not verified. API and component tests provide the recorded evidence.
- The session-race regression has a recorded pre-fix failure and post-fix pass. Pre-implementation failure output for the original financial rules was not retained, so this report does not claim test-first evidence for those changes.
- The earlier full `bun run check` reported existing baseline diagnostics; only the listed targeted Biome checks passed.
- OQ-01, OQ-04, OQ-06, and OQ-07 remain open as recorded in Part A. Docker/GCP deployment is planned for Lab 5; no cloud deployment is claimed.
- Both students should confirm the contribution summary before submission.